

XRP Robotics Learner Manual

30 September 2026 · Edward Gentle

About this manual

This manual takes you from your first drive to confident, well-engineered XRP programs. Work through Chapters 1 to 8 in order; Chapters 9 to 12 cover add-ons and can be done in any order; Chapters 13 to 15 turn a working programmer into a skilled one.

Who it is for: learners who are new to the XRP. You do not need any coding experience.

Contents

Part	Chapters
Foundations	1 Getting started · 2 The Board · 3 Driving and motors · 4 Servo · 5 Sensors
Thinking in code	6 Loops, logic, maths and text · 7 Variables and lists · 8 Functions
Add-ons	9 OLED screen · 10 Device libraries · 11 Remote control · 12 Mecanum wheels
Mastery	13 Using blocks strategically · 14 Projects and challenges · 15 From blocks to Python
Reference	16 Troubleshooting · Glossary

How each chapter works

1. **Learn:** a short explanation of the idea.
2. **Block reference:** every block in the category, pictured as it appears on screen in dark mode, with what it does.
3. **Worked examples:** complete programs you can build and test.
4. **Challenges:** tasks at three levels, so everyone is stretched.
5. **Check yourself:** questions to test your understanding.

Challenge levels

Level	What it asks of you
Bronze	Build what the chapter showed, with small changes
Silver	Combine ideas from this chapter with earlier ones
Gold	Plan and solve a new problem, then explain your choices

Conventions used in this manual

You see	It means
Bold words	A button, menu or block, for example Connect or drive straight
<code>code text</code>	A value you type, or Python code the blocks create
Safety note	Something that protects you, the robot or the computer. Always read these.
Tip	A shortcut or a better way of doing something

Safety first: keep the robot on the floor or on a large table away from the edge, keep fingers, hair and cables away from wheels while a program runs, and switch the robot off before changing wiring.

Chapter 1: Getting started

By the end of this chapter your robot will drive, turn and stop on command, and you will know your way around XRPBlocks, including the difference between **Run** and **Deploy**.

1.1 Meet the XRP

The XRP (Experiential Robotics Platform) is a small robot you program yourself. Its main parts are:

Part	What it does
Controller board	The robot's brain. It stores and runs your program in MicroPython.
Two drive motors with encoders	Turn the wheels. The encoders count how far each wheel has turned.
Motor ports 3 and 4	Two more motor ports, for an arm, a lift or a four-wheel robot
Distance sensor	Measures how far away the nearest object in front is, in centimetres
Line sensor	Two small sensors looking down at the floor, measuring how dark it is
Gyro (IMU)	Measures turning and tilting, so the robot knows which way it faces
Servo ports 1 to 4	Control servo motors, for an arm, a gripper or a flag
LED, RGB LED and user button	A green light, a colour light and a button you can use in programs
Qwiic connector	A four-wire plug for extra devices such as a screen or a colour sensor

Safety note: handle the board by its edges, and never plug or unplug wires while the robot is switched on.

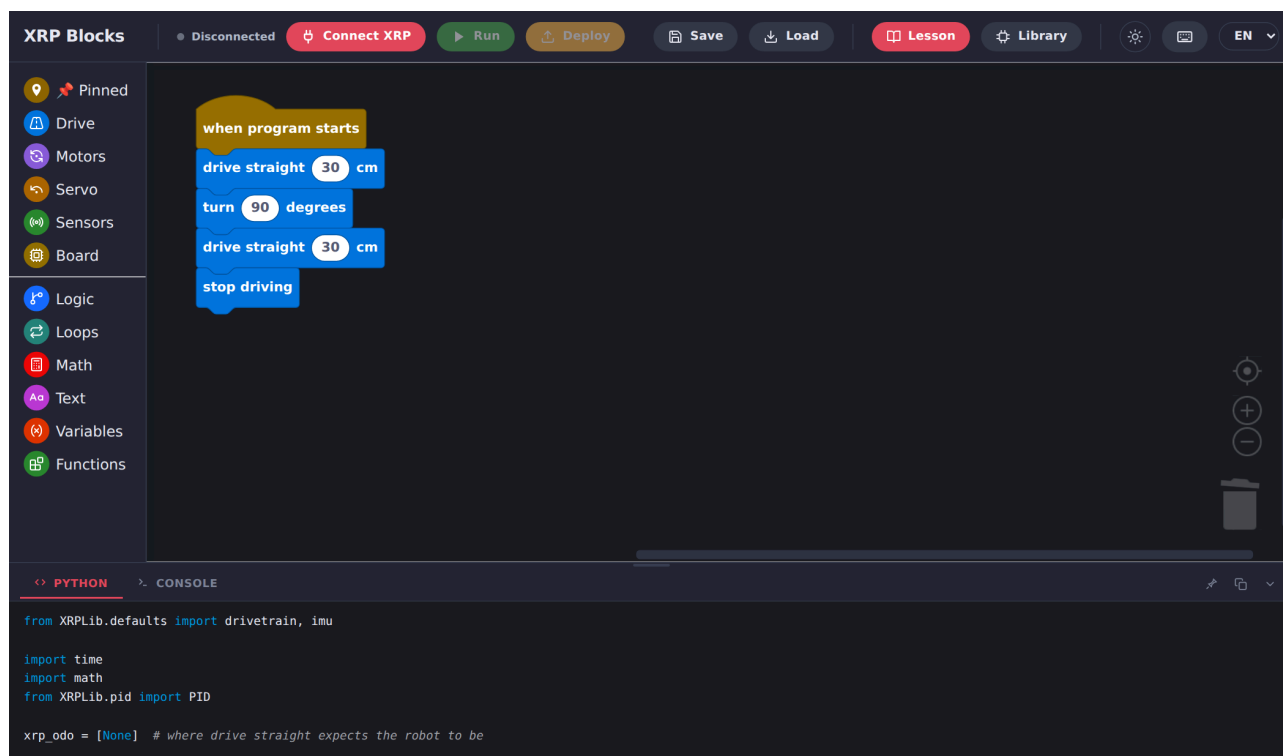
1.2 Power

1. Fit charged batteries into the battery holder. Your teacher will tell you which batteries your kit uses.
2. Switch the robot on with the power switch on the controller board.
3. Switch it off when you are not testing, so the batteries last.

Tip: weak batteries cause strange behaviour, such as short turns, jerky driving or a robot that restarts when a servo moves. Check the batteries first when something odd happens.

1.3 Open XRPBlocks

1. Open the **XRPBlocks** folder and double-click **XRP Robotics.exe**.
2. XRPBlocks opens in your web browser. Use **Google Chrome** or **Microsoft Edge**, because other browsers cannot talk to the robot.
3. Click the sun or moon button to switch between light and dark mode. This manual shows dark mode.



Area of the screen	What it is for
Toolbar (top)	Buttons for the robot and your files (see below)
Toolbox (left)	The block categories. Click one to open its blocks.
Workspace (middle)	Where you drag blocks to build your program
Python tab (bottom)	The real Python code your blocks create
Console tab (bottom)	Messages from the robot, including anything you print and error messages

1.4 The toolbar

Button	What it does
Connect XRP	Connects to the robot by USB cable (or Bluetooth)
Run	Sends your program to the robot and runs it now

Button	What it does
Stop	Stops the running program. It appears while connected.
Deploy	Saves your program on the robot as <code>main.py</code> , to run by itself at power-on
Save / Load	Saves your program to a file, or opens a saved one
Lesson	Opens a step-by-step lesson file from your teacher
Library	Adds device libraries: extra blocks for add-on parts (Chapter 10)
Sun / moon	Light or dark mode
Keyboard	The list of keyboard shortcuts
EN	The language: English or Dutch

1.5 The toolbox

Category	What its blocks do
Pinned	Your favourite blocks. Right-click any block and choose Pin to toolbox .
Drive	Move the whole robot: drive, turn, stop
Motors	Control one motor at a time
Servo	Move a servo to an angle
Sensors	Read the distance sensor, line sensor, button and gyro
Board	Lights, waiting and printing
Library categories	Appear here after you add a device library
Logic	if, compare, and, or, not
Loops	Repeat blocks
Math	Numbers and calculations
Text	Words and messages
Variables	Named values and lists
Functions	Your own named blocks

1.6 Connect the robot

1. Plug the robot into the computer with a USB cable and switch it on.

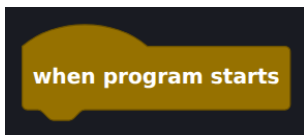
2. Click **Connect XRP** and choose the robot from the list.
3. The status changes from **Disconnected** to connected, and **Run** becomes available.

Next time, XRPBlocks tries the last robot you used, so connecting is quicker. Connecting does not stop a program that is already running on the robot.

Tip: if the robot is not in the list, try a different USB cable. Some cables only charge and cannot carry data.

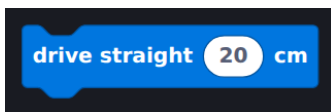
1.7 Your first program

Every program starts with the **when program starts** block. It is already on the workspace and cannot be deleted.

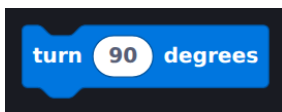


Blocks only run if they are snapped underneath it. Loose blocks turn grey and are ignored.

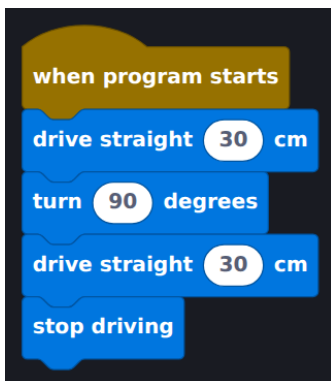
1. Click **Drive** in the toolbox.
2. Drag **drive straight 20 cm** and snap it under **when program starts**. Change 20 to 30.



3. Add **turn 90 degrees**, another **drive straight**, and **stop driving**.



Your program should look like this:



4. Put the robot on the floor with space around it, and click **Run**.

1.8 Run, Stop and Deploy

Button	What happens	When to use it
Run	Sends the program to the robot and runs it now	While you are building and testing
Stop	Stops the program	When something goes wrong
Deploy	Saves the program on the robot as <code>main.py</code> . It does NOT run it now.	When the program is finished and must run on its own

After Deploy, the program runs every time the robot is switched on, even without a computer. XRPBlocks asks you to confirm first, because Deploy replaces whatever program was saved on the robot before.

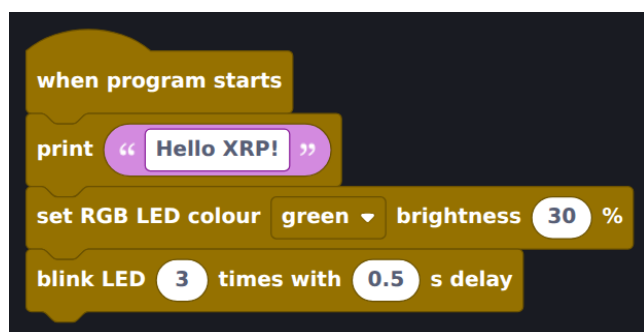
1.9 Reading the Console

Open the **Console** tab while a program runs. Every **drive straight** and **turn** writes one report line there, for example how far the robot really drove and how far off the turn finished. These reports are also saved on the robot, and the last three runs are printed at the start of the next program, so you can read what happened even when the robot ran without a cable.

If the program crashes, the last lines of the Console name the problem.

1.10 Messages and lights

Printing a message and changing a light are good ways to check that your program has started.



1.11 Working with blocks

To do this	Do this
Add a block	Drag it from the toolbox onto the workspace

To do this	Do this
Delete a block	Drag it back to the toolbox or the bin, or select it and press Delete
Copy a block	Select it and press D , or use copy and paste
Undo a mistake	Ctrl + Z
See a block's help	Hover over it: a tooltip explains it
Hide a block's inside	Ctrl + click to collapse or expand it
Switch a block off without deleting it	Alt + click
Tidy up	Right-click the workspace and choose Delete unused blocks
Find your program	Press Home to fit all blocks on the screen

1.12 Keyboard shortcuts

Shortcut	Action
Ctrl + Enter	Run
Ctrl + . (full stop)	Stop. It works even while you are typing.
Ctrl + Shift + Enter	Deploy
Ctrl + S	Save the program to a file
Ctrl + `	Show or hide the Python and Console panel
Ctrl + / or ?	Show all shortcuts
W	Watch the selected variable (Chapter 7)

1.13 Lessons

Your teacher may give you a lesson file. Click **Lesson** and open it: a panel shows the steps, and the toolbox may show only the blocks you need for that lesson. Use **Previous** and **Next** to move through the steps.

1.14 Save your work

- Click **Save** (or Ctrl + S) to download your program as a file.
- Click **Load** to open a saved program.

- Give files clear names, for example `grade8-square-drive.json`.
- Save a new version before a big change, so you can go back.

Challenges

- **Bronze:** make the robot drive forward 50 cm, turn around, and come back to where it started.
- **Silver:** drive an L shape and finish facing the same direction as you started.
- **Gold:** measure how far off your robot finishes after the Bronze task, using the Console reports and a ruler. Explain where the difference comes from.

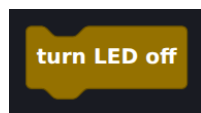
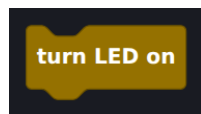
Check yourself

1. Why does a block that is not connected to **when program starts** turn grey?
2. You clicked **Deploy** but the robot did nothing. Why, and what should you do?
3. Where can you see how far the robot really drove?
4. Which key stops a program even while you are typing?

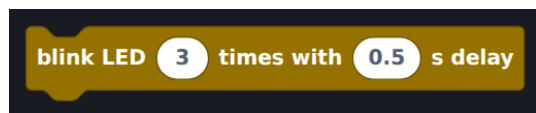
Chapter 2: The Board

The **Board** category controls the lights on the controller, pauses your program and prints messages. These are the blocks you will use most for testing: a light or a message tells you exactly where a program has got to.

2.1 Block reference



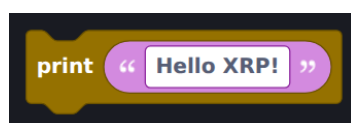
Switch the small green LED on or off.



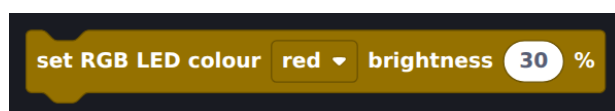
Flashes the green LED a number of times. The program waits while it blinks, so a long blink holds everything else up.



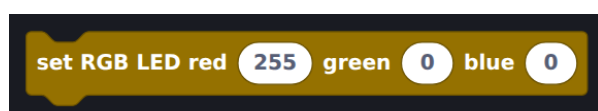
Pauses the program. Decimals work: 0.5 is half a second, 0.02 is two hundredths of a second.



Sends a message to the **Console** tab. Plug in a sensor block to print a reading instead of text.



Chooses one of eleven colours (red, orange, yellow, green, cyan, blue, purple, magenta, pink, white, off) and a brightness from 0 to 100%. 0% switches it off. Values outside 0 to 100 are limited.



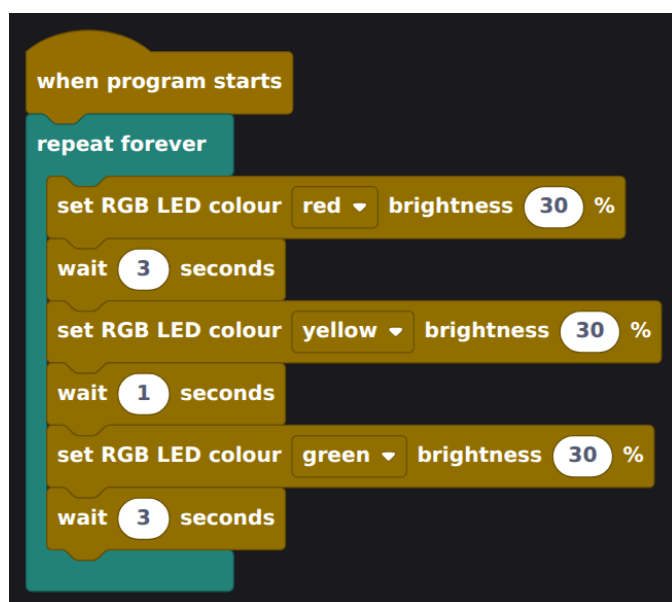
Mixes your own colour from red, green and blue, each from 0 to 255. The RGB LED is not on the older XRP Beta board.

2.2 How colour mixing works

A screen or RGB LED makes every colour from three lights: red, green and blue. Each is set from 0 (off) to 255 (full).

Red	Green	Blue	Colour you see
255	0	0	Red
255	255	0	Yellow
0	255	255	Cyan
255	0	255	Magenta
255	255	255	White
0	0	0	Off

2.3 Worked example: traffic light



The **repeat forever** loop (Chapter 6) keeps the light cycling until you click **Stop**.

2.4 Using lights and prints to find mistakes

When a program does not do what you expect, add a **print** or a colour change at important points, for example "turning now" before a turn. Run it and read the Console: the last message you see tells you the last step that ran. Remove the extra blocks when the problem is fixed.

Challenges

- **Bronze:** change the traffic light so green lasts 5 seconds and the light shows orange instead of yellow.
- **Silver:** make a heartbeat: the RGB LED goes bright red, then dim red, twice quickly, then pauses for one second, forever.
- **Gold:** make the RGB LED fade smoothly from off to full brightness and back, using a **count with** loop (Chapter 6) to change the brightness a little at a time.

Check yourself

1. Your traffic light changes too fast to see. Which block fixes this?
2. What red, green and blue values make yellow?
3. Why is printing messages useful when you look for a mistake?

Chapter 3: Driving and motors

The **Drive** category moves the whole robot; the **Motors** category controls one motor at a time. Drive blocks fall into two groups, blocks that finish by themselves and blocks that keep going, and knowing which group a block belongs to prevents most driving mistakes.

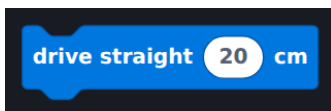
3.1 How the robot steers

The XRP has two driven wheels and no steering wheel. It steers by turning its wheels at different speeds:

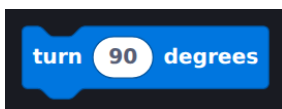
Left wheel	Right wheel	The robot
Same speed forward	Same speed forward	Drives straight
Faster	Slower	Curves to the right
Slower	Faster	Curves to the left
Forward	Backward, same speed	Spins on the spot to the right

This is called **differential drive**, and it is how tanks and many robot vacuums steer.

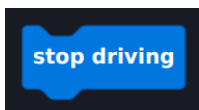
3.2 Drive blocks that finish by themselves



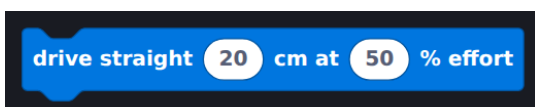
Positive drives forward, negative drives backward. The program waits until the robot arrives.

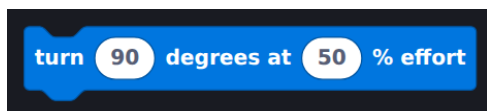


Turns on the spot. Positive turns right, negative turns left. The program waits until the turn is done.



Stops the robot immediately.

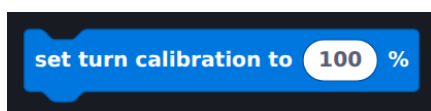




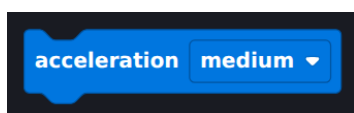
The same two, with the power you choose from 15 to 100%. Less power is gentler and more accurate; more power is faster.

Behind the scenes: drive straight uses the gyro to keep the robot pointing the right way, and makes up for small errors left by the previous drive. **turn** checks the gyro when it stops and nudges the robot, up to 20 times, until it is within half a degree. If a move cannot finish in its time limit (for example the robot is stuck against a wall), it gives up and says so in the Console.

3.3 Tuning blocks

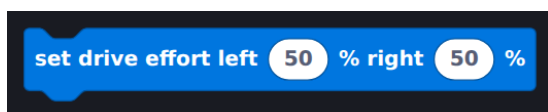


Put this at the start if turns are always slightly too far or too short. 100 means no change. Chapter 13 shows how to work out your number.

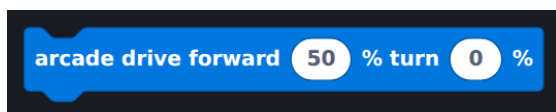


How quickly the robot speeds up and slows down for every movement after this block: extra slow, slow, medium, fast or extra fast. Choose slow if the wheels slip on a smooth floor, fast on a grippy one.

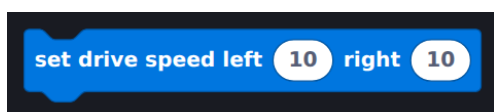
3.4 Drive blocks that keep going



Powers each wheel directly, from -100% (full reverse) to 100% (full forward). The robot keeps going until you stop it.



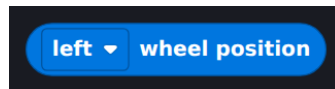
Forward speed and turning in one block, both from -100 to 100%. A positive turn runs the right wheel faster than the left.



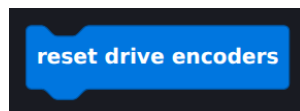
Holds a steady speed for each wheel in **centimetres per second**, using the encoders to correct itself. Useful when the robot must move at the same pace on any battery.

Safety note: a keep-going block with no stop after it drives the robot until you click **Stop**. Always plan how the robot will stop.

3.5 Measuring with the encoders



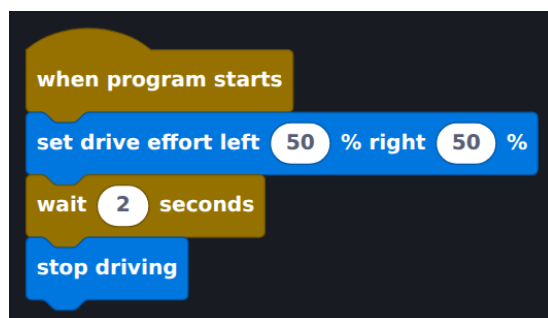
How far the left or right wheel has travelled, in centimetres, since the last reset.



Sets both wheel positions back to zero.

3.6 Worked examples

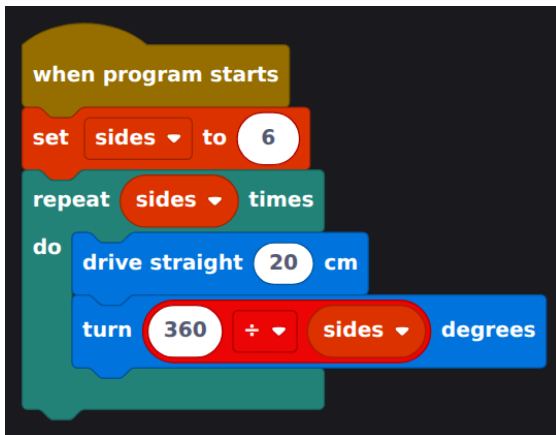
Keep going, then stop. A keep-going block always needs a wait and a stop after it:



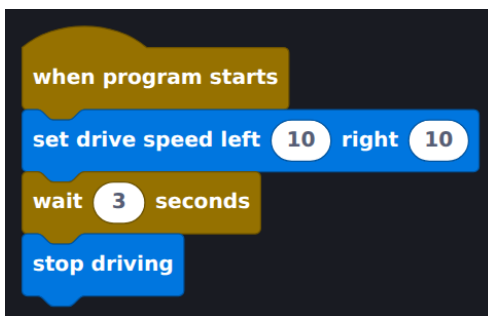
A square. Four equal sides and four equal turns, so a loop does the work:



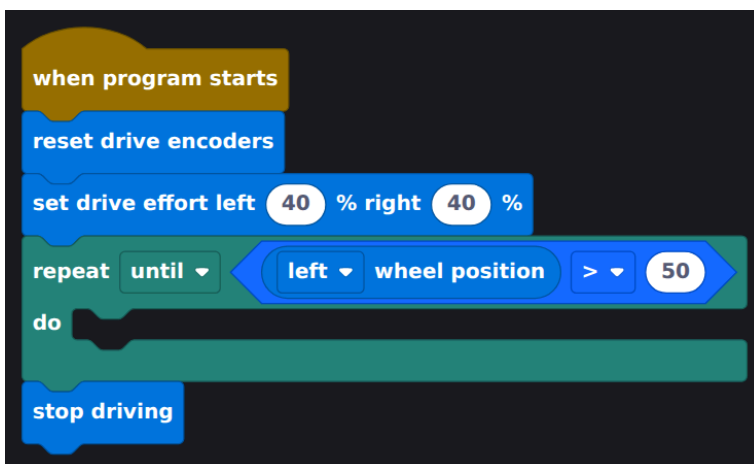
Any shape. For a regular shape with any number of sides, the robot turns 360 divided by the number of sides at each corner. Change sides to 3, 5 or 8 and the same program draws a triangle, pentagon or octagon:



Steady speed. Drive at 10 cm/s for 3 seconds. How far should the robot go? Measure it.

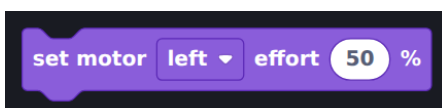


Stop at a distance using the encoders. Start the wheels, then wait until the left wheel has gone more than 50 cm:

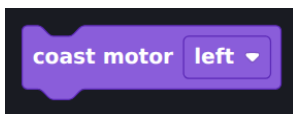
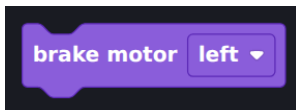


3.7 Motors reference

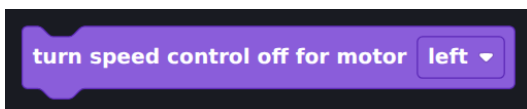
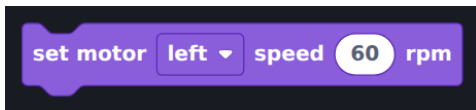
Each Motors block has a list to choose the motor: **left**, **right**, **motor 3** or **motor 4**.



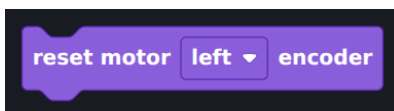
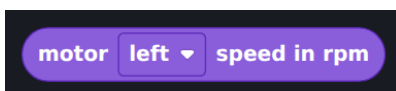
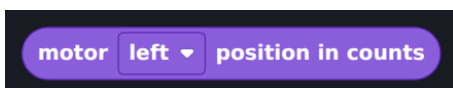
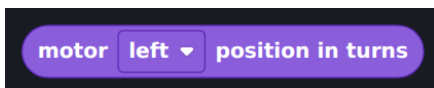
Power for one motor, from -100 (full reverse) to 100 (full forward). 0 stops it.



brake stops the motor and holds it still; **coast** lets it roll to a stop freely.



Holds a speed in revolutions per minute using the encoder, until you switch speed control off. Note that single motors use rpm, while **set drive speed** uses cm/s.



The encoder readings: whole wheel turns, raw counts (a much finer measurement), and the real speed in rpm.

3.8 The maths of wheels

One full turn of a wheel moves the robot by the wheel's **circumference**:

$$\text{circumference} = \pi \times \text{diameter}$$

The standard XRP wheel is about 6 cm across, so one wheel turn moves the robot about $3.14 \times 6 \approx 18.8$ cm. This is how the robot converts encoder counts into centimetres. Measure your own wheel to check.

Challenges

- **Bronze:** drive a triangle. Hint: the robot must turn 120 degrees at each corner, not 60.

- **Silver:** use arcade drive to drive a smooth circle, then stop after one lap. Adjust the time until the robot finishes where it started.
- **Gold:** make the robot drive exactly 1 metre using only **set drive effort** and the encoder, not **drive straight**. Then compare its accuracy with **drive straight 100 cm**.

Check yourself

1. Which blocks in this chapter keep the robot moving until you stop it?
2. How many degrees does the robot turn at each corner of a hexagon (6 sides)?
3. What is the difference between **brake** and **coast**?
4. Your robot overshoots its turns. Name two things you could change.

Chapter 4: Servo

A servo is a small motor that moves to an exact angle and holds it there. Use it for an arm, a gripper, a gate, a flag or a pointer.

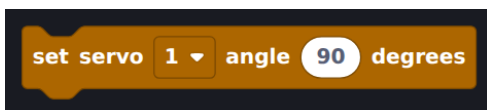
4.1 How a servo works

Inside a servo are a motor, gears and a sensor that measures the output angle. The servo keeps comparing where it is with where it was told to be, and drives the motor until they match. This idea, measuring and correcting, is called **feedback**, and it is also how the robot holds a steady speed and finishes its turns accurately.

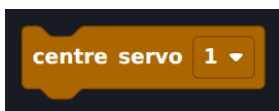
The XRP has servo ports 1 and 2; ports 3 and 4 are on the XRP Controller only.

4.2 Block reference

Every Servo block has a list to choose the servo port, 1 to 4.



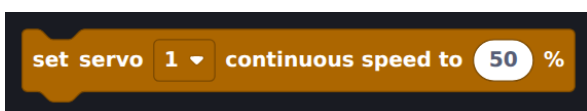
Moves to an angle from 0 to 180 degrees; 90 is the middle. The last 5 degrees at each end are held back, so the servo is never pushed hard against its own end stop.



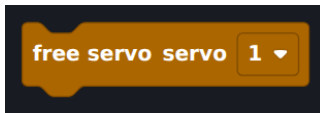
Moves straight to the middle, 90 degrees.



Moves smoothly from one angle to another over a set time, instead of jumping. The program waits until the sweep is finished.



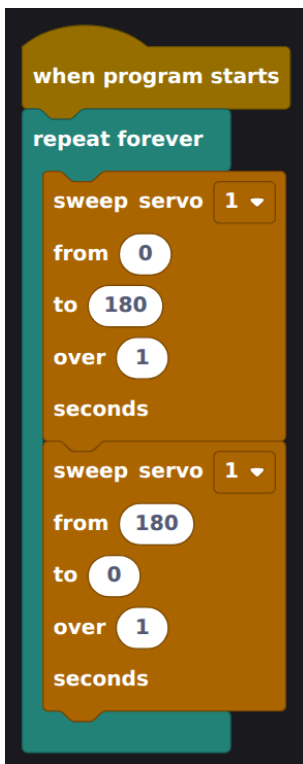
For a **continuous rotation** servo only, which spins like a wheel: -100 is full speed one way, 0 is stop, 100 is full speed the other way. A normal servo just moves to an angle instead.



Stops holding the position, so the servo can be turned by hand and uses less power.

4.3 Worked examples

A sweeping pointer:



A gate that opens on a button press:



4.4 Power and servos

A servo draws a burst of current when it starts to move. If a servo moves while the wheels are driving, the batteries may not keep up: the robot can restart or the gyro reading can jump. Move the servo while the robot is standing still where you can, and keep batteries charged.

Safety note: keep fingers out of an arm or gripper while a program runs. A servo is strong enough to pinch.

Challenges

- **Bronze:** build a waving arm that waves three times when the program starts, then returns to the middle.
- **Silver:** make a pointer that sweeps slowly from 10 to 170 degrees while the RGB LED changes colour at the halfway point.
- **Gold:** build a boom gate for a model car park: the gate opens when something comes closer than 10 cm to the distance sensor (Chapter 5), stays open for 3 seconds, then closes.

Check yourself

1. What angle is the middle position?
2. What is the difference between **set servo angle** and **sweep**?
3. Why might the robot restart when a servo moves while it drives?

Chapter 5: Sensors

Sensors let the robot react to the world instead of following a fixed script. This chapter covers the four built-in sensors and introduces **proportional control**, the idea behind smooth, accurate robot movement.

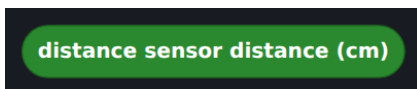
5.1 Reading block shapes

The shape of a block tells you how it fits:

Shape	Gives	Examples
Jigsaw notch top and bottom	Nothing: it does an action	drive straight, print
Round ends (reporter)	A number or text	distance sensor, gyro yaw
Pointed ends (Boolean)	true or false	button is pressed, compare

A reporter or Boolean block does nothing on its own. Plug it into another block that uses the value.

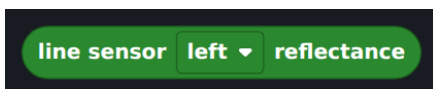
5.2 Distance sensor



Reports how far away the nearest object in front is, in centimetres. The sensor sends out a short pulse of sound, too high for people to hear, and times how long the echo takes to come back, like a bat.

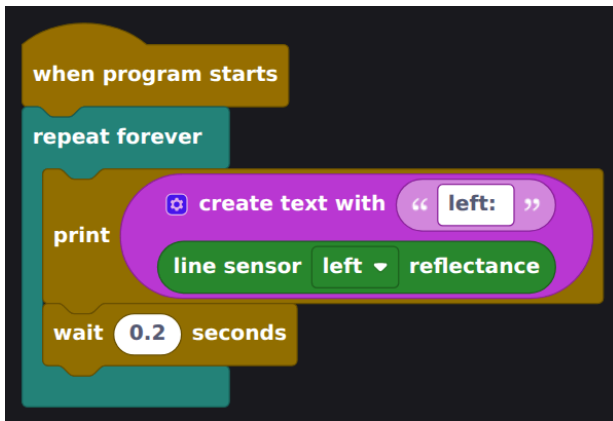
Good to know: hard, flat surfaces facing the sensor give the best readings. Soft things (clothing, curtains) and surfaces at an angle can scatter the sound, and very close or very far objects may give odd values. Print the readings and test the range with your own objects before relying on them.

5.3 Line sensor



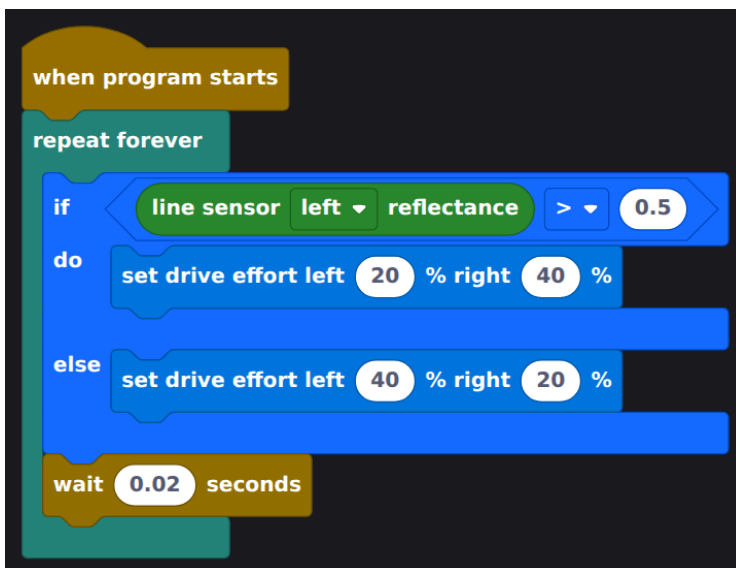
Two small sensors under the robot shine light at the floor and measure how much bounces back. Each reports from 0 for white to 1 for black. It is a decimal, not a percentage.

Calibrate first. Floors, tape and lighting differ, so measure your own:



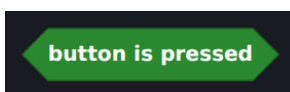
1. Run this with the sensor over the white floor. Write down the reading.
2. Move it over the black line. Write down that reading.
3. Your **threshold** is halfway between. For example, white 0.1 and black 0.9 give a threshold of 0.5.

A simple line follower steers towards the side that sees the line:

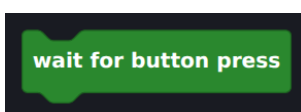


This version follows the **edge** of the line: it zig-zags along the border between black and white. Replace 0.5 with your own threshold.

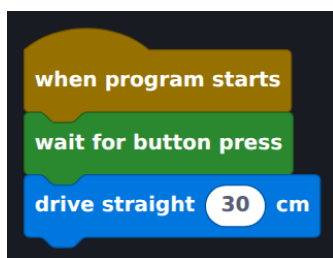
5.4 The user button



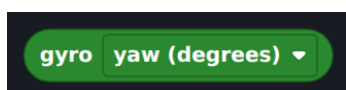
true while the button on the board is held down.



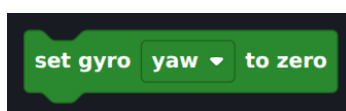
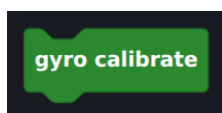
Holds the program until you press the button. Use it so the robot does not drive off the moment you click **Run** or switch it on:



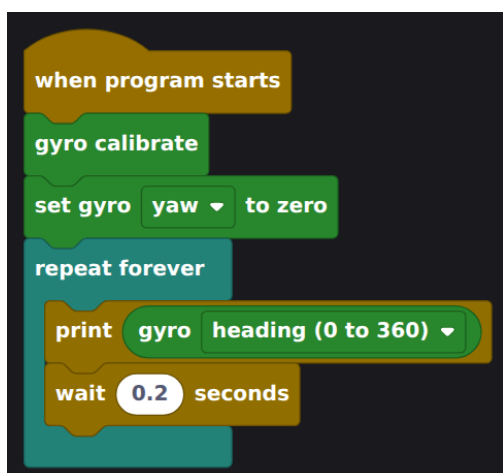
5.5 Gyro (IMU)



Reading	What it measures
yaw	Turning left and right. Keeps counting past 360 (two turns right is 720).
heading	Turning, but wraps from 0 to 360 like a compass
pitch	Tilting nose up or down
roll	Tilting to the left or right side

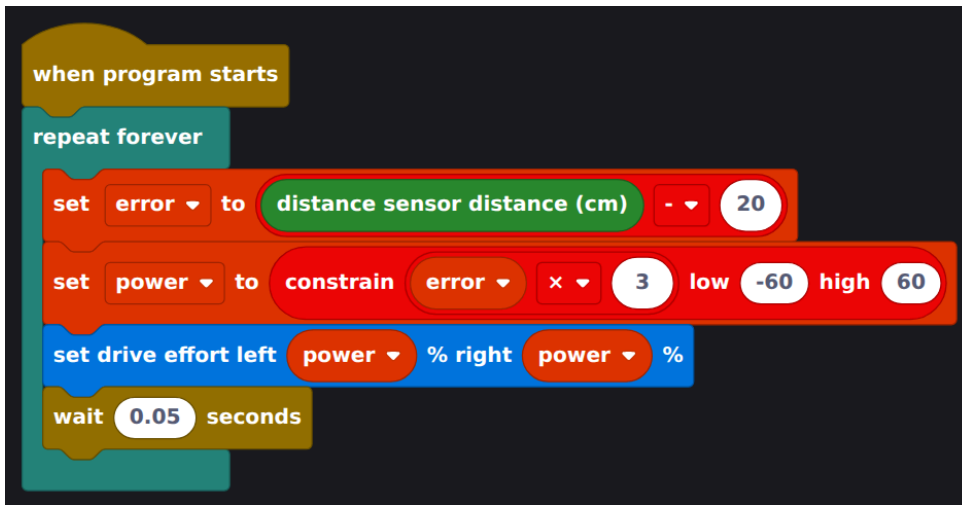


Always **calibrate** first with the robot completely still, then set the angle you care about to zero (the list offers yaw, pitch or roll). The gyro starts counting the moment the robot powers up, so without this the angle is already a large number before your program starts.



5.6 Proportional control: react in proportion

A simple **if** gives only two answers: go or stop. Better robots react **in proportion** to how wrong things are: a big error gets a big correction, a small error a small one. This program keeps the robot 20 cm from your hand:



Hand at	$\text{error} = \text{distance} - 20$	$\text{power} = \text{error} \times 3$	The robot
40 cm	20	60	Drives forward quickly
25 cm	5	15	Creeps forward
20 cm	0	0	Stops
10 cm	-10	-30	Backs away

The 3 is called the **gain**. Too small and the robot is lazy; too big and it overshoots and wobbles. **constrain** keeps the power within safe limits. This same idea is used in real self-driving cars, drones and factory robots.

Challenges

- **Bronze:** print the distance sensor reading in a forever loop. Find the closest and furthest distances your sensor measures reliably.
- **Silver:** calibrate your line sensor on your own track and make the line follower complete a full lap.
- **Gold:** tune the gain in the follow program. Try 1, 3 and 8, describe what happens with each, and explain why.

Check yourself

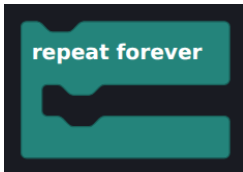
1. What value does the line sensor give on white paper, roughly?
2. Why must the robot be still while the gyro calibrates?
3. What is the difference between yaw and heading?

4. In the follow program, what does the robot do when the error is negative?

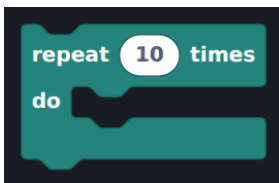
Chapter 6: Loops, logic, maths and text

Loops repeat blocks, logic makes decisions, maths calculates and text builds messages. Together they turn a list of instructions into a robot that thinks.

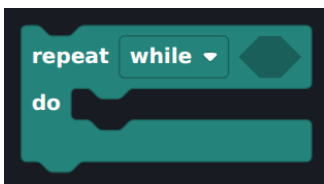
6.1 Loops



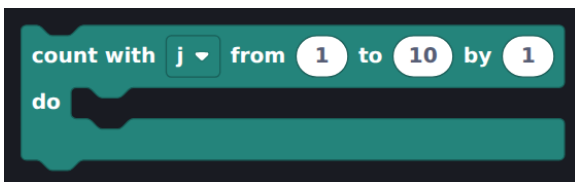
Runs the blocks inside over and over until you click **Stop**. Nothing can be snapped underneath it, because it never ends.



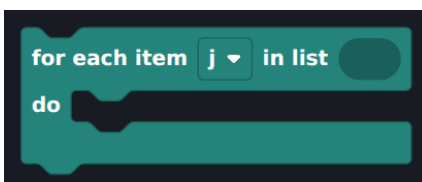
Repeats a fixed number of times. You can plug a variable into the number.



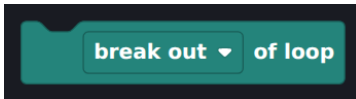
Repeats **while** a condition is true, or (switch the list) **until** it becomes true. If the condition is already met at the start, the blocks inside never run.



Repeats once for each number, storing the current number in the variable *i*. Use *i* inside the loop, for example to drive further each time.



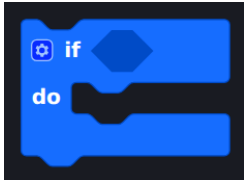
Repeats once for each item in a list (Chapter 7).



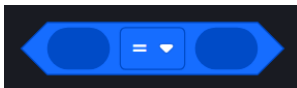
break out leaves the loop straight away; **continue with next iteration** skips to the next pass.

Loops inside loops. Put a **repeat 4** (a square) inside a **repeat 3** and the robot draws three squares. The inner loop finishes completely on each pass of the outer loop.

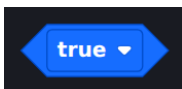
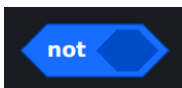
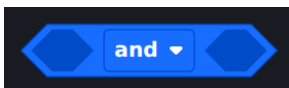
6.2 Decisions



if runs the blocks inside only when its condition is true; **else** runs when it is false. Click the cog on an **if** block to add **else if** parts, for choices with more than two answers.



Compares two values: = , ≠ , < , ≤ , > or ≥ .

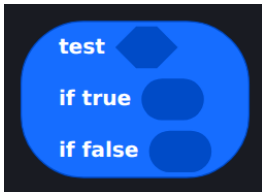


Combine conditions:

A	B	A and B	A or B
true	true	true	true
true	false	false	true

A	B	A and B	A or B
false	true	false	true
false	false	false	false

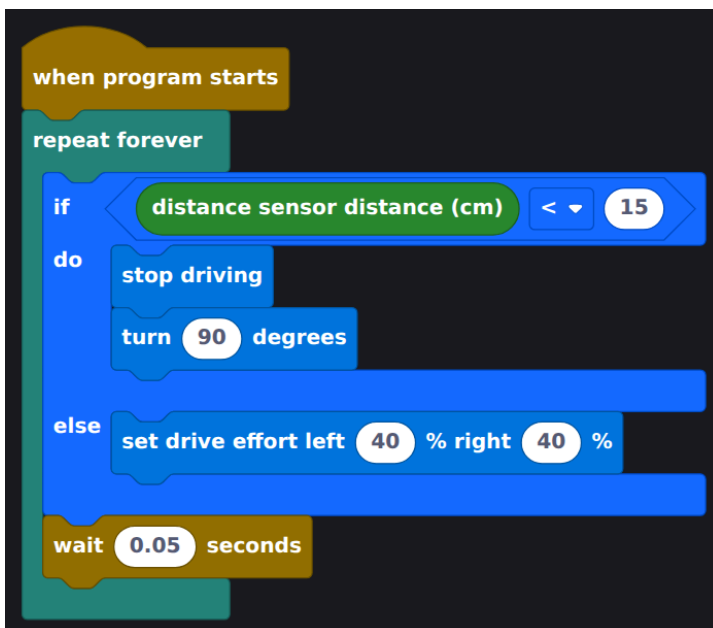
not flips true to false and false to true.



Gives one of two values depending on a test, all in one block. For example: test distance < 20 , if true "close" , if false "far" .

6.3 Worked examples

Avoid obstacles:

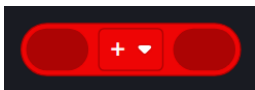


Every 0.05 seconds the robot checks the distance. Closer than 15 cm, it stops and turns; otherwise it keeps driving.

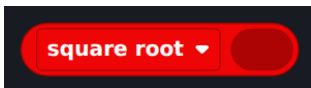
Two conditions at once: red only when something is close **and** the button is held.



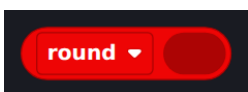
6.4 Maths



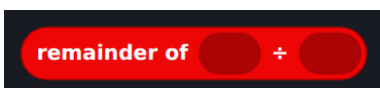
$+$, $-$, $\times$, $\div$ and $^$ (to the power of).



Square root, absolute value (drops the minus sign), negative, and logarithms.



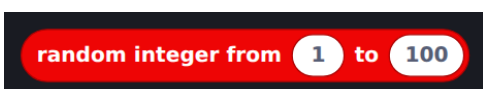
Round, round up or round down.



The remainder after dividing. remainder of $7 \div 3$ is 1. Useful for "every third time": when remainder of $\text{count} \div 3 = 0$.



Keeps a number between a low and a high limit. Use it to keep motor power safe.



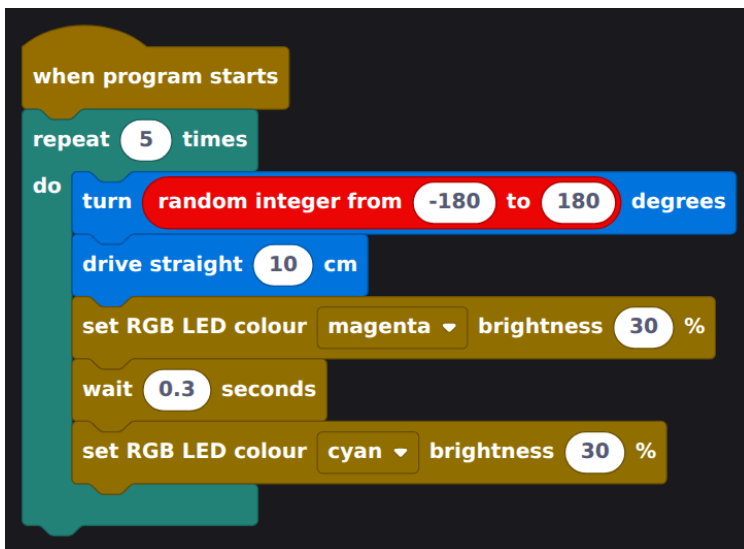
A random whole number between two limits, including both. **random fraction** gives a decimal between 0 and 1.



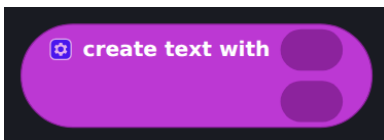
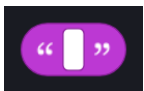
Checks whether a number is even, odd, prime, whole, positive, negative or divisible by another.

The Math category also has **sin**, **cos** and **tan** (in degrees) and constants such as π .

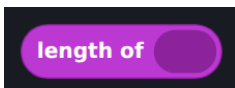
A random dance:



6.5 Text



create text with joins pieces into one message. Click the cog to add more pieces. It is the easiest way to print a labelled reading, such as left: 0.83 (see Chapter 5).



Other Text blocks count letters, check whether text is empty, find one piece of text inside another, and pick out a single letter.

Challenges

- **Bronze:** draw three squares using a loop inside a loop.
- **Silver:** make the RGB LED green when nothing is near, yellow between 10 and 30 cm, and red closer than 10 cm, using **else if**.
- **Gold:** make the robot drive a spiral: each side is 5 cm longer than the last. Use **count with** and plug `i` into the distance.

Check yourself

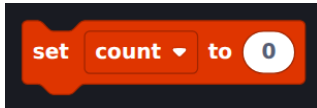
1. Why is there a short **wait** inside the forever loop of the obstacle avoider?
2. Which loop would you use to drive exactly five laps? Which to drive until the button is pressed?
3. What is remainder of $10 \div 4$?
4. If A is true and B is false, what is A **or** B?

Chapter 7: Variables and lists

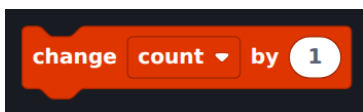
A variable is a named box that holds one value; a list is a named box that holds many values in order. Both live in the **Variables** category, and both let a program remember things.

7.1 Variables

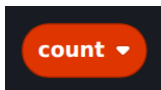
Click **Make a Variable** and give it a clear name, such as `count`, `speed` or `obstacle limit`.



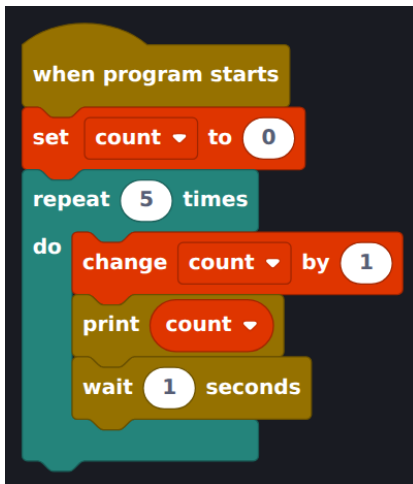
Puts a new value in the box, replacing the old one.



Adds to the value. Use a negative number to subtract.



Gives you the value, to plug into another block.



Good habits with variables:

- Always **set** a variable to a starting value before you use it.
- Name it after what it holds, not `a` or `x`.
- A variable can be used anywhere in the program, including inside functions.

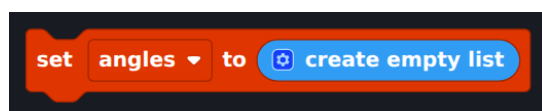
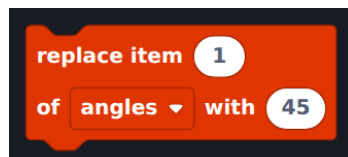
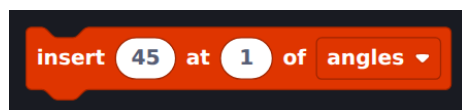
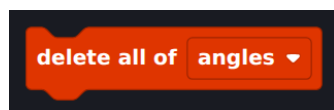
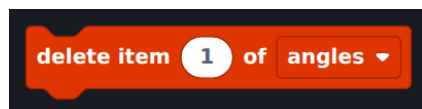
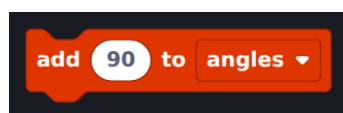
7.2 Watching a value while the robot runs

Right-click a variable or list block and choose **Watch this value** (or select it and press **W**). Its round block glows. Click **Run**, then hover over the round block: a tooltip shows its current value on the robot and how long ago it changed. Watching only works with **Run**; a deployed program is never slowed down by it.

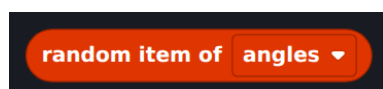
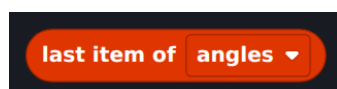
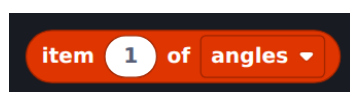
7.3 Lists

Click **Make a List** (below **Make a Variable**) and name it, for example `angles`. A new list starts empty; you do not need to set it first. Items are numbered from **1**, like Scratch.

Adding and removing:



Reading:



length of angles ▾

Checking:

angles ▾ contains 90 ?

is angles ▾ empty?

The whole list, and text:

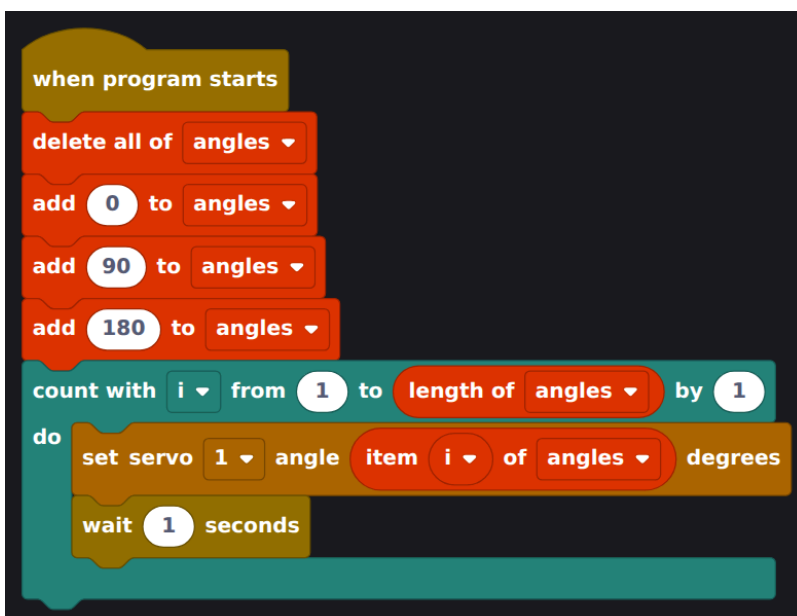
angles ▾

join angles ▾ with “ , ”

make list from “ 0,90,180 ”
split by “ , ”

The last two turn a list into text with a separator (for example 0,90,180) and back again.

7.4 Worked example: a list of servo angles



```
when program starts
  delete all of angles ▾
  add 0 to angles ▾
  add 90 to angles ▾
  add 180 to angles ▾
  count with i ▾ from 1 to length of angles ▾ by 1
  do
    set servo 1 ▾ angle item i ▾ of angles ▾ degrees
    wait 1 seconds
```

The **count with** loop runs *i* from 1 to the length of the list, and **item *i* of angles** picks each angle in turn. Add more angles and the program still works without any other change. This is the power of lists.

Important: when a list holds numbers, plug a number block (from **Math**) into **add**, as in the picture. The block normally offers a text box, and text that looks like a number is still text. See Chapter 13.

Challenges

- **Bronze:** make a list of four distances and drive each one in turn, turning 90 degrees after each.
- **Silver:** record the distance sensor reading every second for 10 seconds into a list, then print the list and its largest value.
- **Gold:** make a route recorder: a list of drive distances and a list of turn angles, played back by one loop. Change the route by changing only the lists.

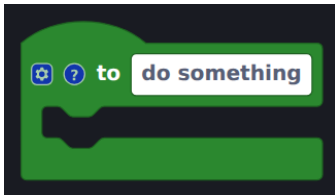
Check yourself

1. What is the number of the first item in a list?
2. What is the difference between **set** and **change**?
3. Why is a list better than ten separate variables for ten readings?
4. How can you see a variable's value while the robot is running?

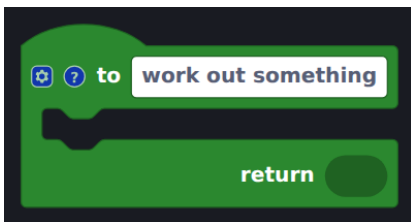
Chapter 8: Functions

A function is a set of blocks you name once and then use many times, like teaching the robot a new word. Functions are how programmers break a big problem into small, testable parts.

8.1 Block reference



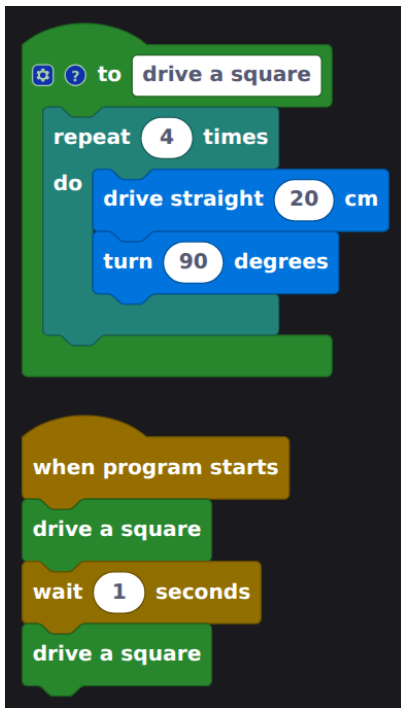
Defines a function that **does** something, such as driving a shape or flashing a warning.



Defines a function that **works out and gives back** a value, such as the average of two readings. Its call block is round, like a sensor block.

8.2 Make and use a function

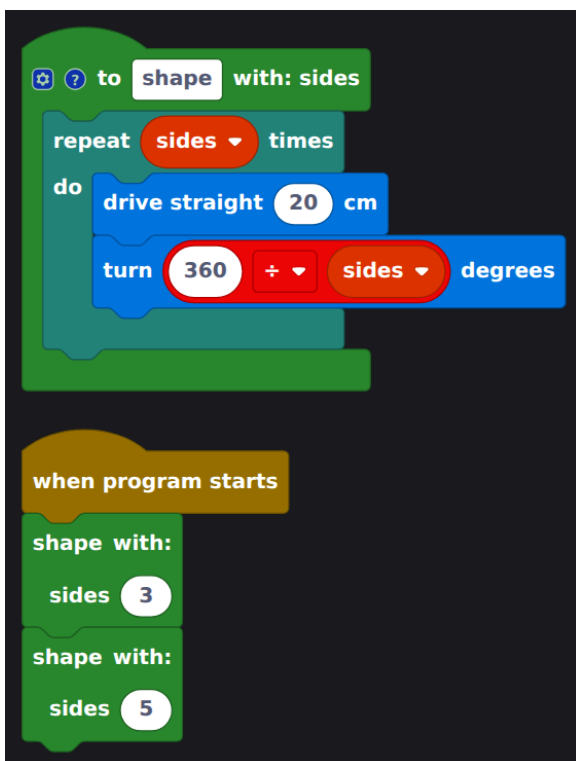
1. Open **Functions** and drag out **to do something**.
2. Click **do something** and type a name, for example `drive a square`.
3. Put the blocks inside it.
4. A new block with that name now appears in **Functions**. Snap it under **when program starts** wherever you want those steps to happen.



The function block sits on its own on the workspace. It is not greyed out, because the program reaches it through the blocks that call it.

8.3 Inputs make functions flexible

Click the cog on a function block and drag in an **input**. The input becomes a variable that only that function uses. This function draws any regular shape:



The call **shape with sides 3** draws a triangle; **shape with sides 5** draws a pentagon. One function, many shapes.

8.4 Breaking a problem down

Before building a big program, write its main steps in plain words, then make each step a function:

Task: deliver a parcel and come back	Function
Wait for the start signal	get ready
Follow the line to the drop-off point	follow line to end
Lower the arm and release	drop parcel
Turn around and return	go home

The main program then reads like the plan: four function calls under **when program starts**. Build and test one function at a time.

Challenges

- **Bronze:** make a function called `flash warning` that blinks the RGB LED red three times, and call it before every turn in your square.
- **Silver:** give `drive a square` an input called `size`, and draw three squares of different sizes.
- **Gold:** write a function that **returns** the average of five distance readings taken 0.05 seconds apart. Use it in the obstacle avoider and compare how steady the robot is.

Check yourself

1. You changed the side length inside `drive a square`. How many places in the program did that change?
2. What is the difference between **to do something** and **to work out something**?
3. Why is it easier to test a program made of functions?

Chapter 9: OLED screen

A small OLED screen lets the robot show words, readings and pictures without a computer. It is 128 pixels wide and 64 pixels high, and plugs into the Qwiic connector.

9.1 How the screen connects

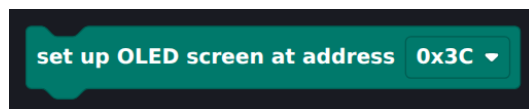
The Qwiic connector uses **I2C**, a way for many small devices to share two signal wires. Each device has an **address**, a number that works like a house number on a street, written in hexadecimal such as `0x3C`. Almost every OLED screen uses `0x3C`; a small link on the back can change it to `0x3D`.

1. Switch the robot off and plug the screen into the Qwiic connector.
2. In XRPBlocks click **Library**, choose **OLED screen**, and add it.
3. A new **OLED** category appears in the toolbox.

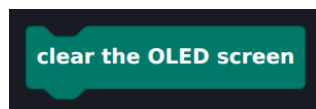
9.2 The screen's grid

Positions on the screen are counted in pixels from the **top-left corner**: `x` runs 0 to 127 across, `y` runs 0 to 63 **down**. So the centre is `x 64, y 32`. For text, the screen has eight lines of 16 characters.

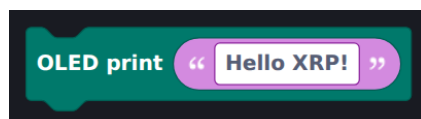
9.3 Text blocks



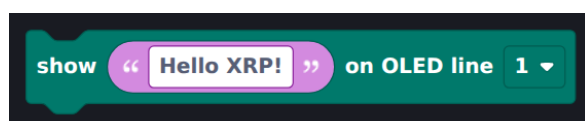
Use once, near the top of the program, outside any loop.



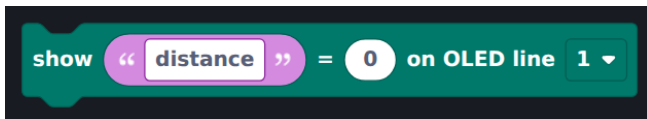
Wipes the screen and starts printing from the top again.



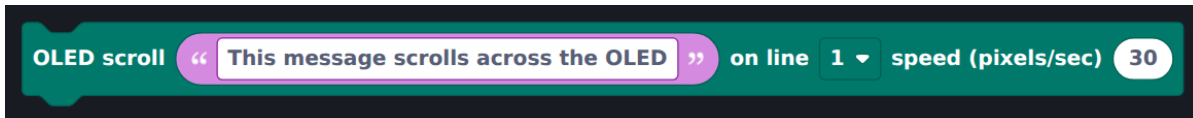
Prints a line at the bottom, like the Console. After eight lines the screen scrolls up.



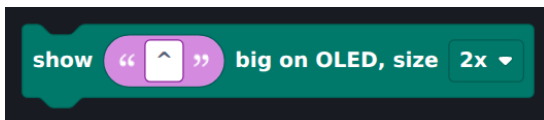
Puts text on one of the eight lines, replacing what was there.



Shows a label and a reading together, such as distance: 24 . Long decimals are rounded to two places.

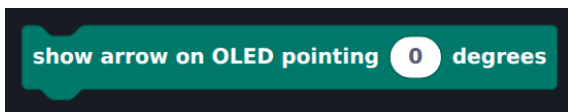


Scrolls long text across one line. Put it inside a forever loop with a short wait; it does not pause your program. Speed is 1 to 120 pixels per second.

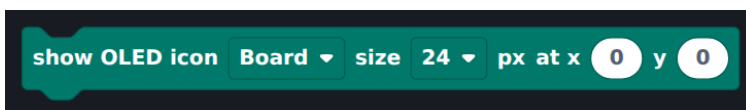


Text blown up 2, 3, 4 or 6 times and centred: good for a countdown you can read across the room.

9.4 Drawing blocks



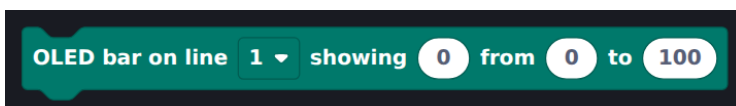
0 points up, 90 right, 180 down, 270 left. Plug the gyro heading in and the arrow works like a compass needle.



Built-in icons, 16, 24 or 32 pixels square. You can also draw your own picture in the OLED designer page (tools/oled-designer.html) and paste it into the **show custom picture** block.

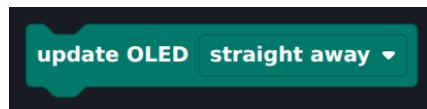


Other drawing blocks: pixel, line, circle, and outline or filled shapes.



A bar across one line that fills as a reading rises, like a battery or distance meter.

9.5 Screen settings

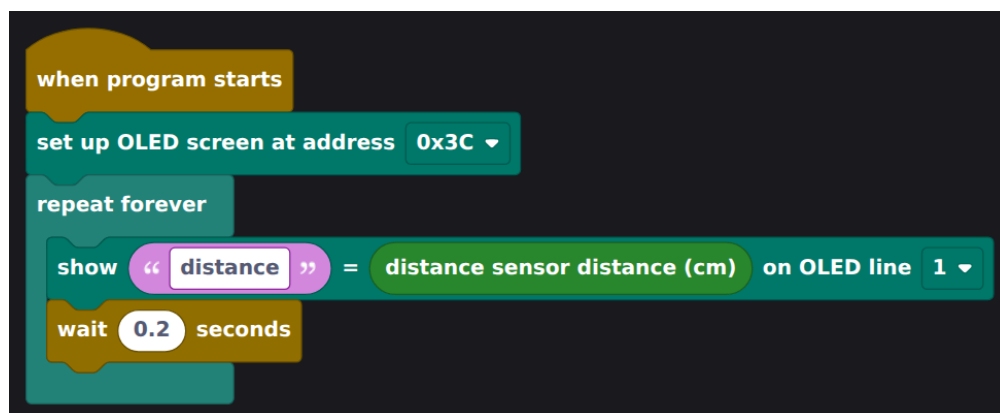


Normally every change appears at once. Choose **only when I say update** and use **update the OLED now** after writing several lines: the screen then changes once, which is much faster and does not flicker.

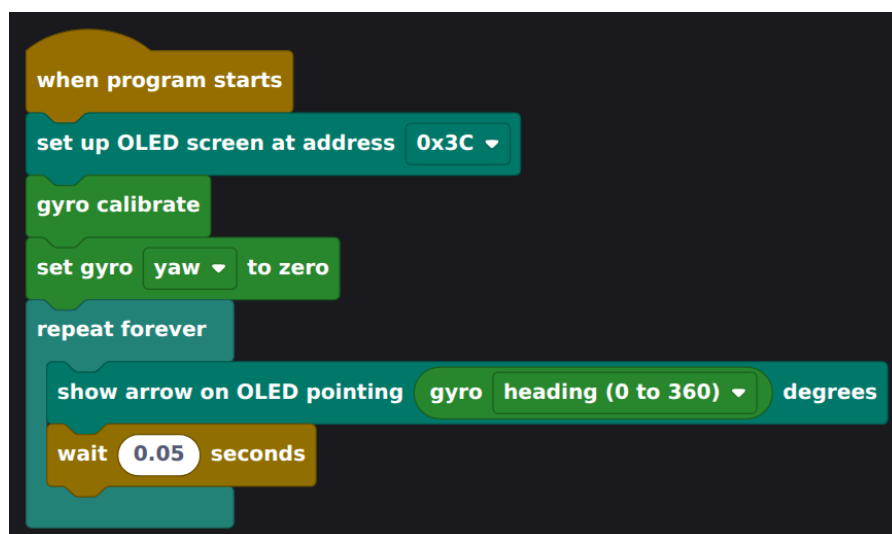
Other settings: brightness (0 to 100%), inverted colours, screen on or off, and **OLED is connected** to check that the screen answers.

9.6 Worked examples

A live distance display:



A compass:



Challenges

- **Bronze:** show your name on line 1 and the distance on line 3.

- **Silver:** make a countdown from 5 to 1 in big digits, then show **GO!** and drive forward.
- **Gold:** build a dashboard with the heading on line 1, the distance on line 2 and a distance bar on line 4, using manual update mode so it does not flicker.

Check yourself

1. How many text lines fit on the screen?
2. Why does **set up OLED screen** go outside the forever loop?
3. Where on the screen is x 0, y 0?

Chapter 10: Device libraries

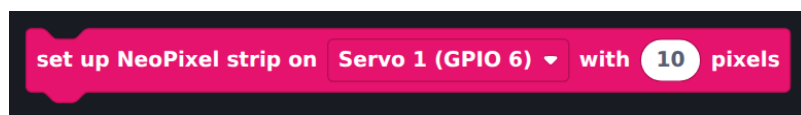
A device library adds blocks for an extra part, such as a light strip, a sensor or a screen. Add one with the **Library** button; its category then appears in the toolbox, and XRPBlocks installs the part's driver (its Python code) on the robot automatically when you click **Run**.

Library	What it is for
NeoPixel strip	A strip of colour lights, each controlled separately
PCF8575 I/O expander	Sixteen extra pins for LEDs and buttons
Colour sensor (TCS34725)	Reads the colour of an object
LCD 16x2 and LCD 20x4	Character screens with two or four lines
Triggers	Automatic lights and actions, such as brake lights and indicators
OLED screen	Chapter 9
Remote control	Chapter 11
Phone control (Bluetooth)	Chapter 11 (11.5 and 11.6)
Mecanum wheels	Chapter 12

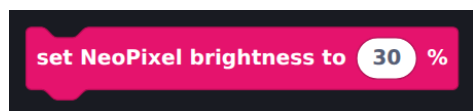
Safety note: switch the robot off before plugging in any add-on part, and check the wiring before switching on.

10.1 NeoPixel strip

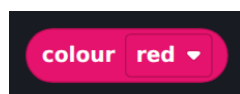
A NeoPixel strip is a chain of tiny RGB LEDs. Each pixel has a number, starting at 0.

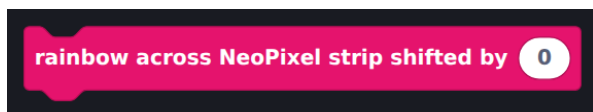
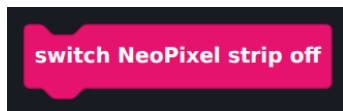
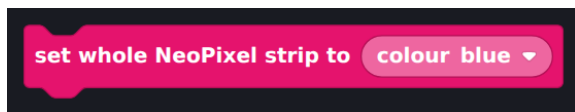
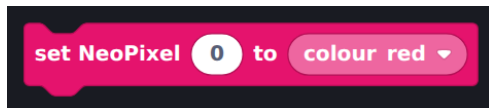


Says which connector the strip is on (a servo port or a free GPIO pin) and how many pixels it has. Use once, at the start.

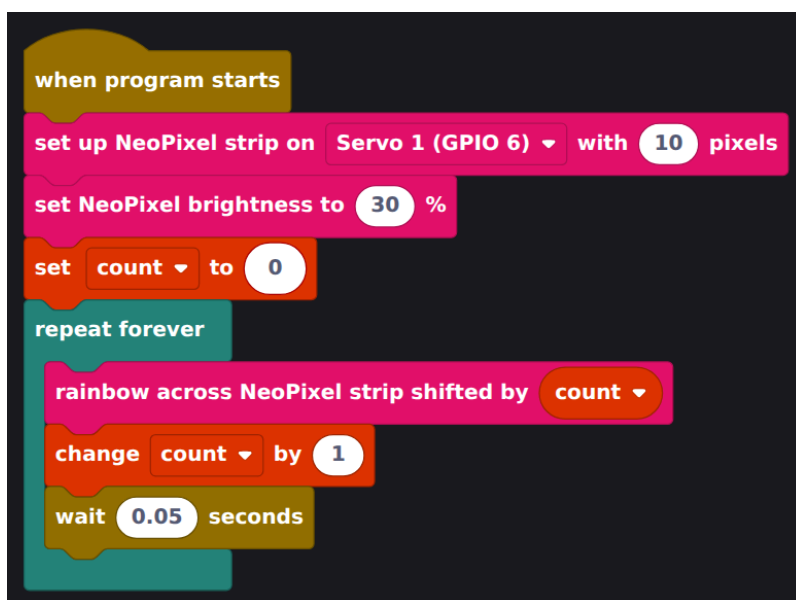


Keep the brightness low: a bright strip draws a lot of current from the batteries.



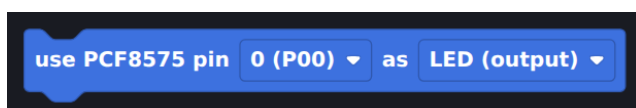
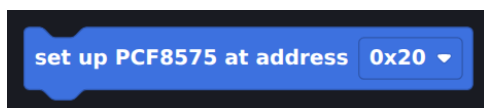


Other blocks shift the whole pattern along the strip, mix a colour from red, green and blue, and report the number of pixels. If updating is set to **only when I say show**, changes appear only after **show NeoPixel changes**, which is faster for big patterns.

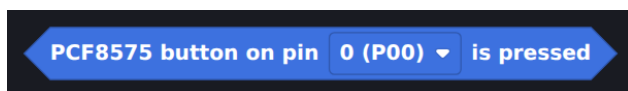
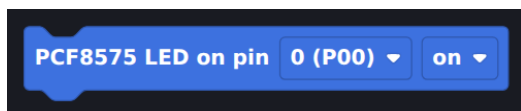


10.2 PCF8575 extra pins

The PCF8575 adds sixteen pins through the Qwiic connector, numbered 0 to 15 (printed on the board as P00 to P17).

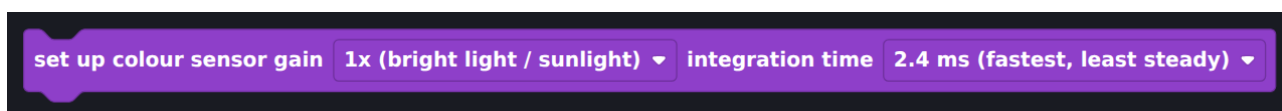


Tell it what is wired to each pin: an **LED** (output) or a **button** (input).

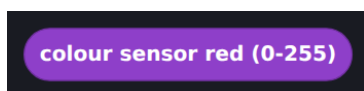


A button is wired between the pin and GND. Other blocks switch all LEDs off, set a pin high or low directly (for a buzzer or relay), and read a pin as 1 or 0.

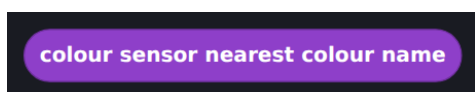
10.3 Colour sensor



Choose the **gain** (1x for bright light up to 60x for very dark) and the **integration time** (2.4 ms, fastest, to 614 ms, steadiest). Higher settings are more sensitive but slower.



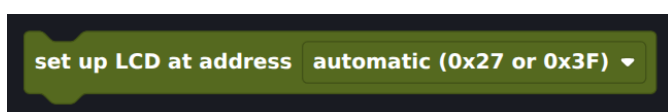
Red, green and blue each from 0 to 255. Each block takes its own reading, a moment apart.



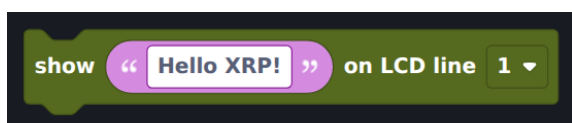
Reports the closest of eight colours: black, white, red, green, blue, yellow, cyan or magenta. It is a learning aid, not an exact measurement: lighting and distance both affect it. Other blocks give a raw light level, an approximate brightness in lux and an approximate colour temperature.

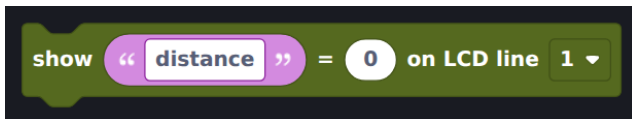
Tip: hold objects at the same distance from the sensor every time, and shade it from room light, for steadier readings.

10.4 LCD screens



automatic tries the two common addresses, 0x27 and 0x3F. The LCD needs 5 V power.

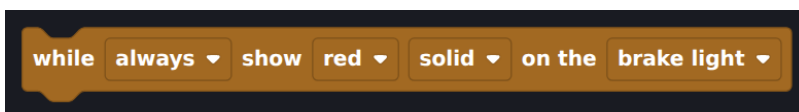
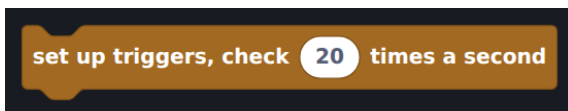




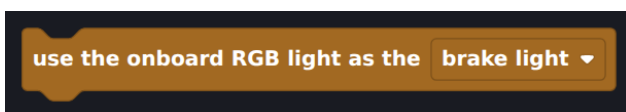
The 16x2 LCD has two lines of 16 characters; the 20x4 has four lines of 20. The blocks work like the OLED text blocks, plus a text progress bar, backlight on or off, and a cursor.

10.5 Triggers: rules that run in the background

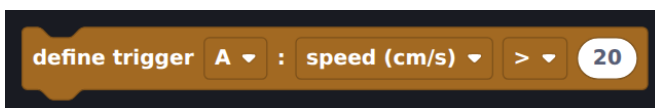
The Triggers library lets you say "whenever this happens, do that" once, at the start. The rules are then checked about 20 times a second while your main program carries on driving.



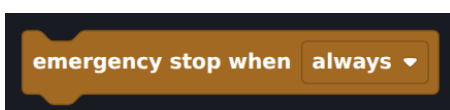
A **light rule**. Events include always, moving, driving forward, reversing, turning left or right, braking, stopped, board button pressed, and your own triggers A to D. Patterns are solid, blinking or pulsing.



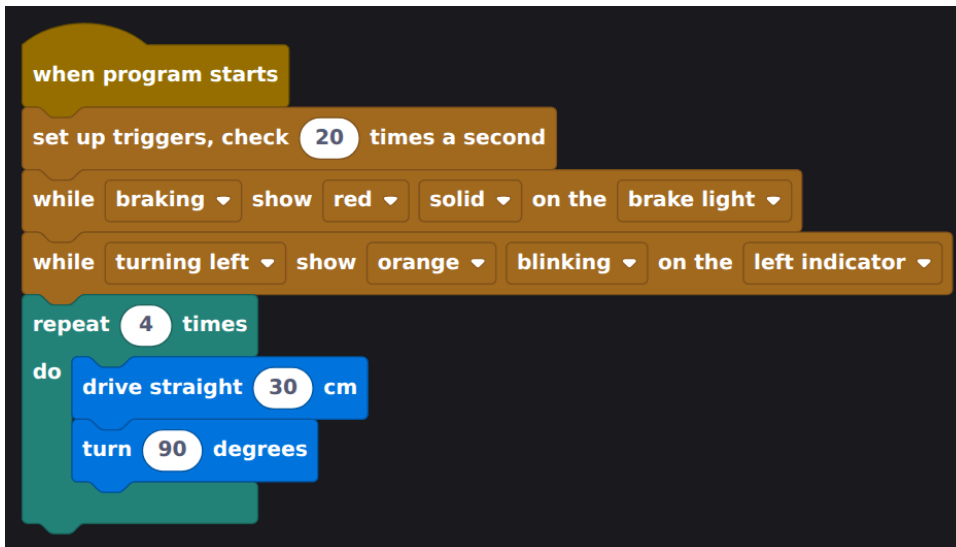
Six lights (brake, reversing, left indicator, right indicator, headlight, status) can each be assigned to the RGB light, the board LED, an LED on a pin, or pixels on a strip. Unassigned lights use the RGB light. Do not also use the Board RGB blocks, or they will fight over the light.



Your own triggers A to D, from a measurement (speed, turning speed, distance travelled, battery voltage, line sensor, seconds since start), a switch on a pin, a value your program posts, or a timer.



An optional safety rule: when it happens, every motor brakes and motor commands are ignored until **clear emergency stop**.



The square is driven by ordinary Drive blocks; the lights look after themselves.

Challenges

- **Bronze:** light a NeoPixel strip in your school colours.
- **Silver:** make a colour sorter: show the colour sensor's nearest colour name on the OLED or LCD, and set the RGB LED to match.
- **Gold:** give your robot a full set of car lights with Triggers: headlights always on, brake light, reversing light and both indicators. Drive a course and check each light works.

Check yourself

1. Which button adds a new device library?
2. What number does the first NeoPixel on a strip have?
3. Why should NeoPixel brightness be kept low?
4. What is the difference between a trigger rule and an **if** block in a loop?

Chapter 11: Remote control

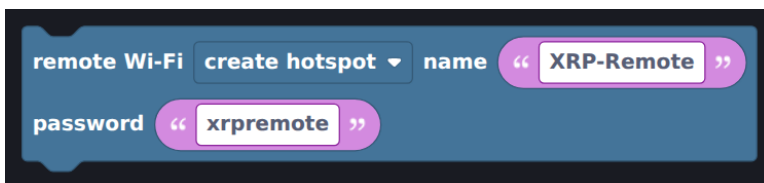
The **Remote control** library turns a phone, tablet or laptop into a controller. The robot runs a small web page with six buttons; your blocks decide what each button does.

Sections 11.5 and 11.6 cover a second library, **Phone control (Bluetooth)**: an Android phone drives the robot with a joystick and no Wi-Fi, and can show the robot's sensor readings and messages.

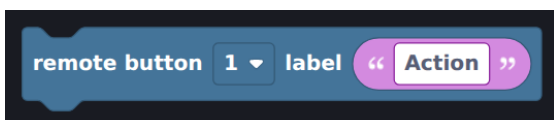
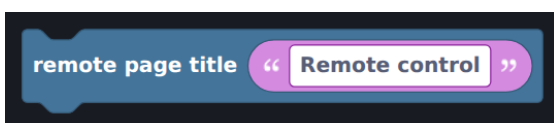
11.1 How it works

The robot acts as a tiny web server. Your phone opens the robot's page in its browser, and every time you hold or release a button the page sends a message to the robot. While a button is held, the page keeps sending short "still here" messages. If the messages stop for one second (the phone went to sleep, or walked out of range), every button reads as released. Only one controller can be in charge at a time.

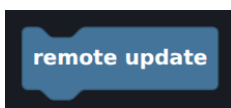
11.2 Block reference



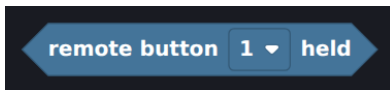
Starts the web page. **create hotspot** makes the robot its own Wi-Fi network (default name XRP-Remote , password xrpremote ; passwords must be 8 to 63 characters). **join network** connects the robot to an existing 2.4 GHz Wi-Fi network instead; it waits up to 30 seconds and prints the address to use.



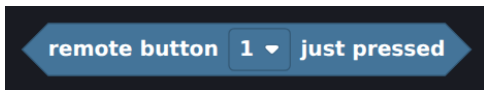
Set the page title and button names **before** starting the remote. Refresh the page to see changes.



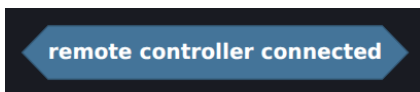
Lets the robot answer the page. Put it first in the forever loop, with a short wait (0.02 seconds) at the end of the loop.



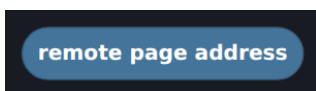
true while the button is held. Use it for driving.



true once per press. Use it for single actions, such as switching a light on and off.



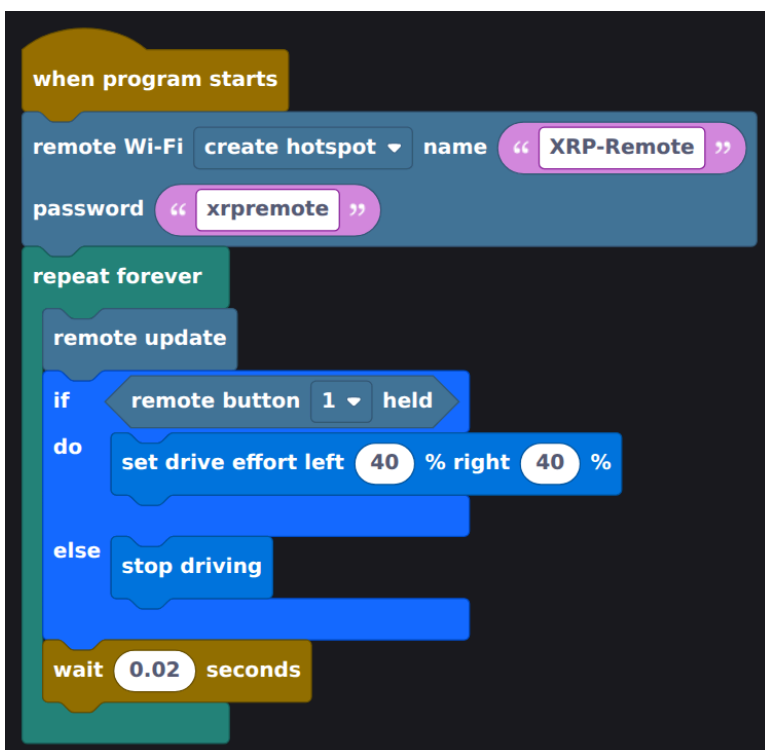
true while a controller is sending messages.



The web address to open. Print it, or show it on the OLED.

Two more blocks **release all buttons** and **close** the remote (switching its Wi-Fi off).

11.3 A first remote program

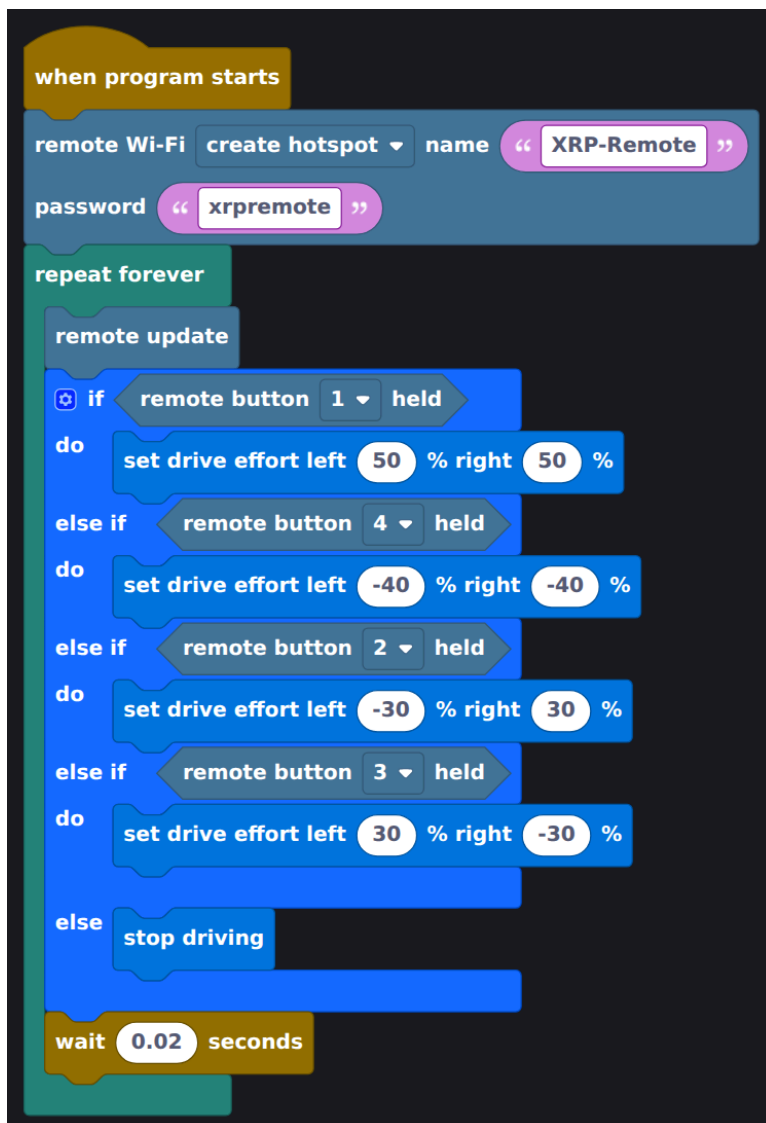


1. Build the program above and click **Run** with the robot on USB.
2. Open the **Console** and note the web address it prints.
3. Connect your phone to the XRP-Remote Wi-Fi. Stay connected even if the phone says there is no internet.

4. Open the address in the phone's browser and hold button 1.

11.4 Driving with the D-pad

On the phone, choose the **Game D-pad** layout. Its buttons are numbered: Up = 1, Left = 2, Right = 3, Down = 4, A = 5, B = 6.



Use the cog on the **if** block to add the **else if** parts. You can rename the buttons on the page itself; the names are remembered in that browser.

Safety note: the remote does not stop the motors by itself when the connection drops. Always put **stop driving** in the **else** part, as in both examples, so the robot stops when nothing is held.

Tip: avoid waiting blocks such as **drive straight 50 cm** inside a remote loop: while the robot waits, it cannot hear the phone.

Privacy: the Wi-Fi password is saved inside your program file. Use a password made only for the robot, never a home or school password.

11.5 Phone control over Bluetooth

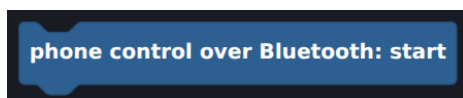
The **Phone control (Bluetooth)** library lets an Android phone drive the robot with a joystick and six buttons, with no Wi-Fi at all. The phone talks to the robot over the same Bluetooth link that XRPBlocks can use to connect.

	Remote control (Wi-Fi)	Phone control (Bluetooth)
Phone	Any phone, tablet or laptop	Android phone with Chrome
Network	Robot hotspot or a 2.4 GHz network	None needed
Page	Served by the robot	Opened from an https:// web address
Controls	Six buttons or D-pad	Joystick, D-pad, six buttons, or Messages
Robot to phone	Nothing	Readings and messages (11.6)

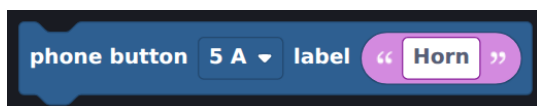
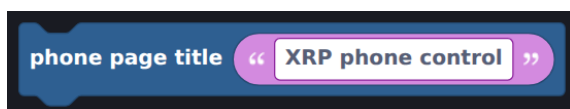
What you need: Chrome on an Android phone, and the address of the phone page your teacher gives you. It must start with **https://**; a page saved on the phone and opened as a file cannot use Bluetooth. iPhones cannot use it, because Safari does not support Web Bluetooth.

One at a time: the phone and XRPBlocks cannot both be connected to the robot by Bluetooth. Connect XRPBlocks with the USB cable instead, or disconnect it first.

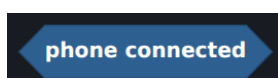
Block reference



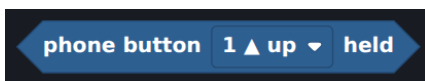
Switches on the robot's Bluetooth link and starts listening for the phone. Put it under **when program starts**.



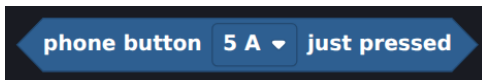
The title and button names the phone shows. Buttons keep their numbers: Up = 1, Left = 2, Right = 3, Down = 4, A = 5, B = 6.



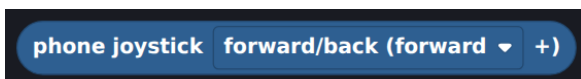
true while the phone is sending. It turns false one second after the phone stops, for example when the screen locks.



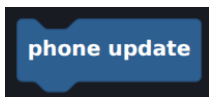
true while the button is held. Use it for driving.



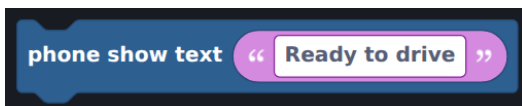
true once per press. Use it for single actions, such as switching a light.



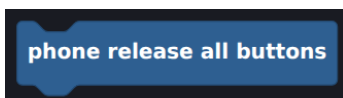
The joystick position from -100 to 100. **forward/back** is positive when pushed forward; **left/right** is positive when pushed right. Both are 0 in the middle or when the phone is not connected.



Reads waiting messages from the phone. The reading blocks above do this for you; you only need it in a loop that reads none of them.

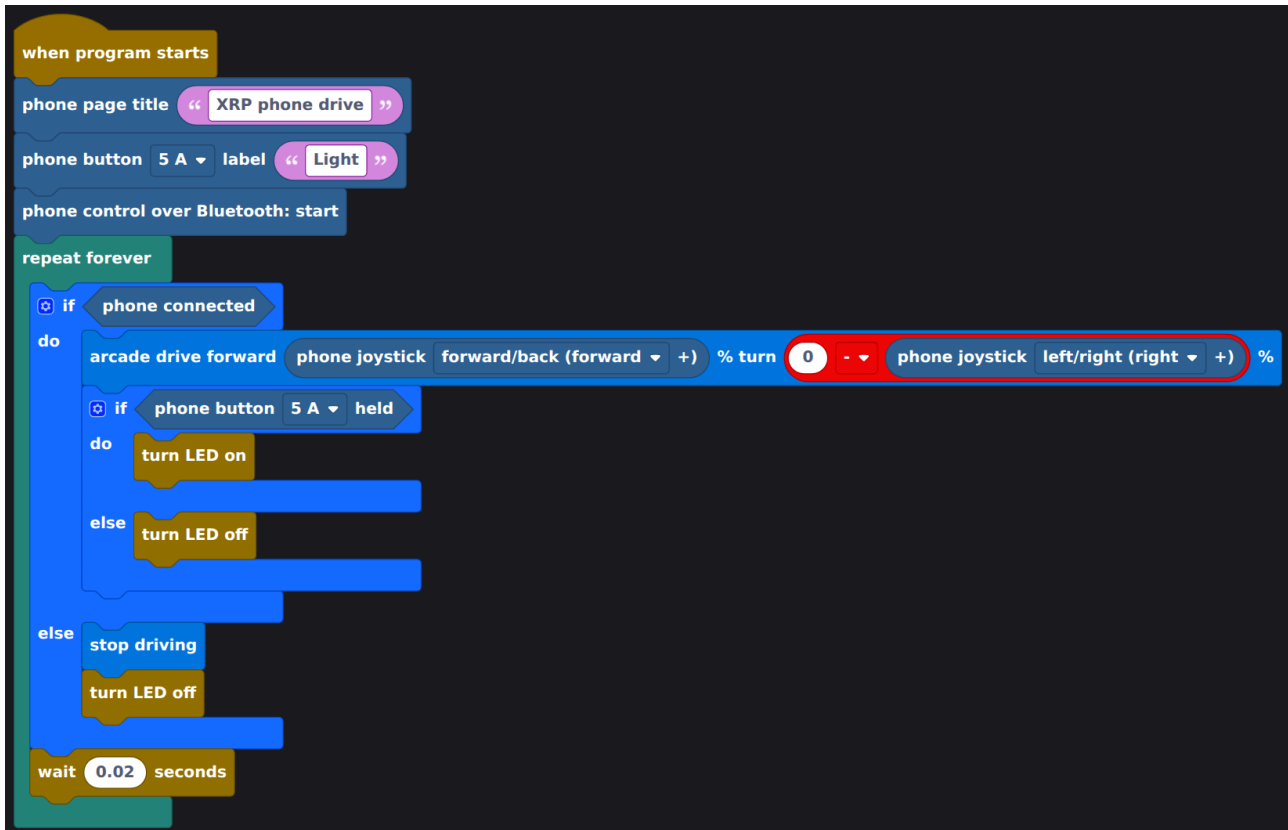


Shows one short line under the title on the phone.

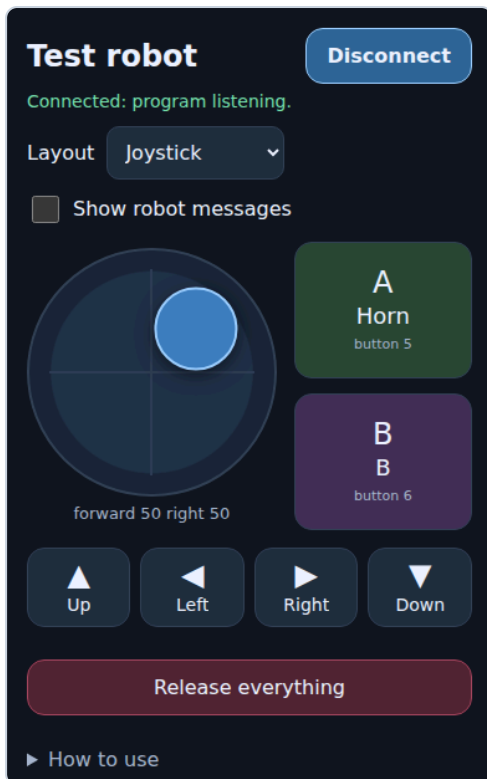


Clears the buttons and joystick. It does not stop the motors.

A first Bluetooth program



1. Build the program above (it is also working program 23) and click **Run** with the robot on USB.
2. On the phone, open the phone page in Chrome and tap **Connect**. Choose the robot; its name starts with **XRP-**.
3. The status line turns green: **Connected: program listening**. Push the joystick to drive; hold **A** for the light.

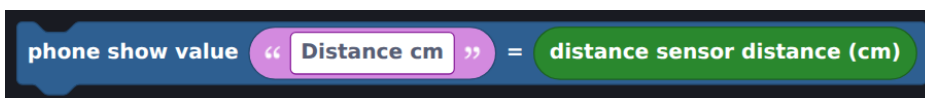


The **arcade drive** block turns left when its turn value is positive, so the program uses **0 minus left/right** to make pushing right turn right. If your robot turns the wrong way, take the **0 minus** out.

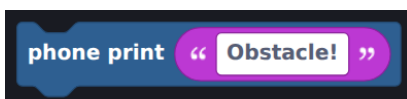
Safety note: just like the Wi-Fi remote, the robot does not stop by itself when the phone goes away. The **else** part with **stop driving** is what stops it, about one second after the phone disconnects.

11.6 Readings and messages on the phone

The phone can also show what the robot is doing: sensor readings as large numbers, and a log of short messages. This is a quick way to debug a program away from the laptop.



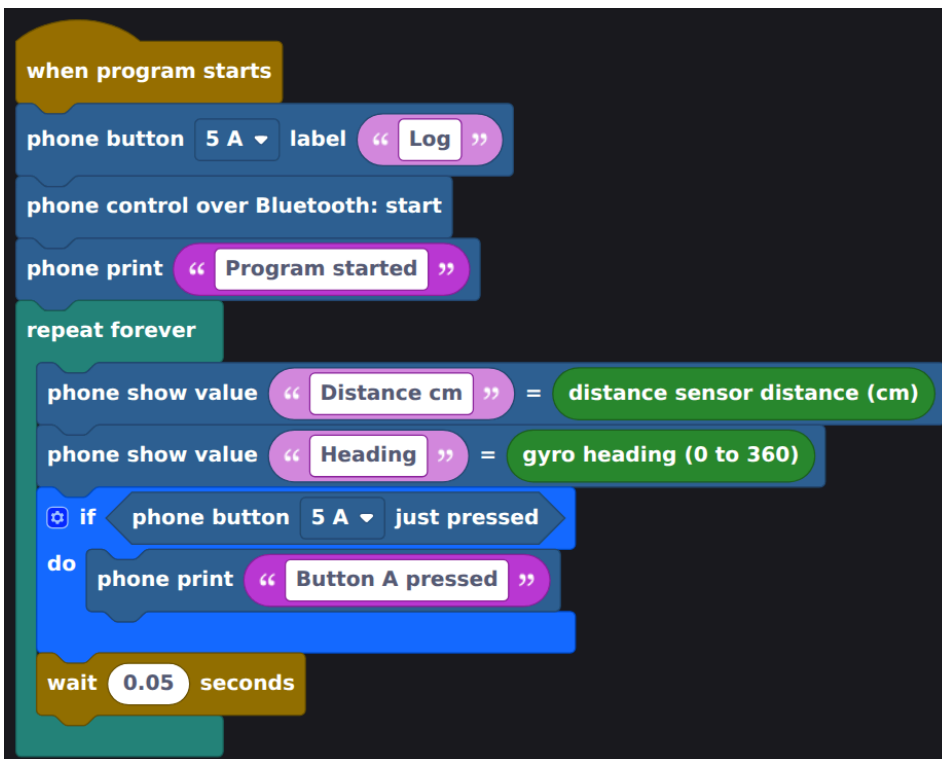
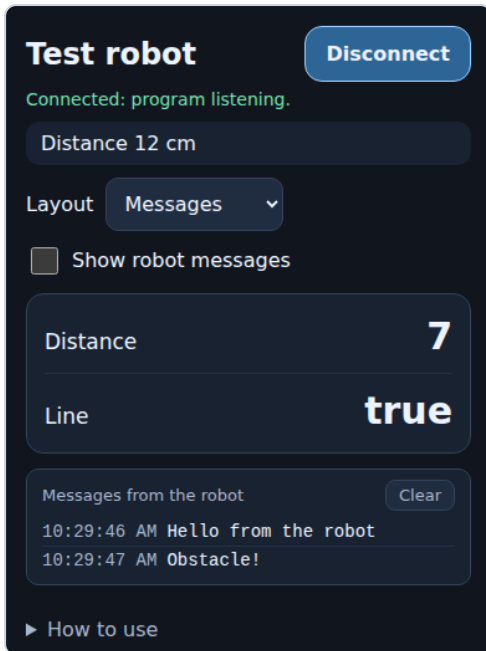
Shows a named reading on the phone, such as the distance sensor. Each name gets its own tile, updated in place. You may use up to 8 names, and decimals are rounded to 2 places. It is safe inside a fast loop: the robot sends changes at most five times a second.



Adds a line to the message log on the phone, with the time it arrived. Use it for events ("Obstacle!", "Lap 3"), not every time round a fast loop.

The phone only shows these when you ask for them, in one of two ways:

- Tick **Show robot messages** under the layout. The tiles appear above the controls and the log below them, so you can drive and watch at the same time.
- Choose **Messages** in the **Layout** list. The controls disappear and the page shows one large block with every reading, with the log underneath. This works whether or not the box is ticked.



This is working program 24. Run it, connect the phone, choose **Messages**, then move your hand in front of the distance sensor and turn the robot by hand. Tap **A** to add a line to the log.

Tip: ordinary **print** blocks still go to the XRPBlocks Console only. Use **phone print** for anything you want to read on the phone.

Challenges

- **Bronze:** make button 5 switch the RGB LED between red and green each time it is pressed.
- **Silver:** add button 6 as a "turbo" button: while it is held together with Up, drive at 80% instead of 50%.
- **Gold:** build a remote-controlled delivery robot: drive with the D-pad, open and close a servo gripper with A and B, and show the connection status on the OLED.
- **Bluetooth Silver:** drive with the phone joystick, and show the distance sensor reading on the phone with phone show value.
- **Bluetooth Gold:** make the robot stop by itself and phone print "Too close!" when the distance drops below 10 cm, even while the joystick is pushed forward.

Check yourself

1. Why must **stop driving** be in the **else** part?
2. What is the difference between **held** and **just pressed**?
3. Which button number is Left on the D-pad?
4. Why must the phone page's address start with https://?
5. What does the robot do when the phone's screen locks, and which part of your program makes it stop?
6. When would you use phone show value, and when phone print?

Chapter 12: Mecanum wheels

Mecanum wheels let a four-wheeled robot slide sideways and diagonally, not only forwards and backwards. This needs four motors and the **Mecanum wheels** library.

12.1 How mecanum wheels work

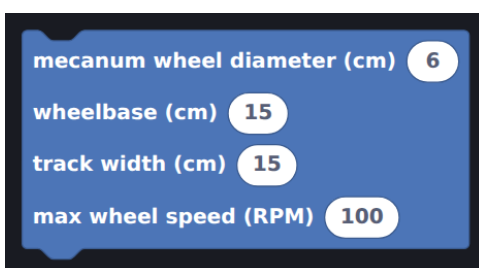
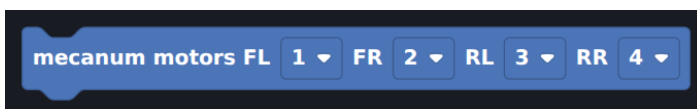
Each mecanum wheel has small rollers around its edge, set at 45 degrees. When the wheel turns, the rollers push the robot partly forward and partly sideways. By turning each of the four wheels a different way, the sideways pushes either cancel out or add up:

Front left	Front right	Rear left	Rear right	The robot
Forward	Forward	Forward	Forward	Drives forward
Forward	Backward	Backward	Forward	Slides right
Backward	Forward	Forward	Backward	Slides left
Forward	Backward	Forward	Backward	Spins right on the spot

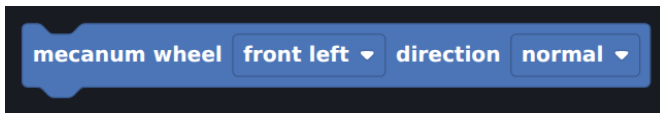
The library works out these combinations for you. It expects the standard layout, where the rollers form an **X** when you look down on the robot.

12.2 Set up

Corner	Default motor port
Front left (FL)	1
Front right (FR)	2
Rear left (RL)	3
Rear right (RR)	4

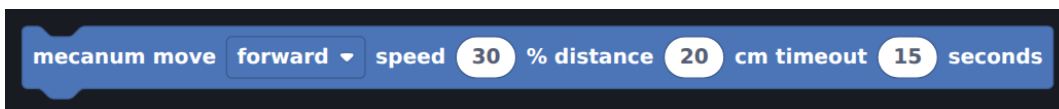


Put both blocks first under **when program starts**. Measure your own robot in centimetres: wheel diameter, **wheelbase** (front axle to rear axle) and **track width** (left wheel centre to right wheel centre). The defaults are only placeholders. **max wheel speed** sets what 100% means.

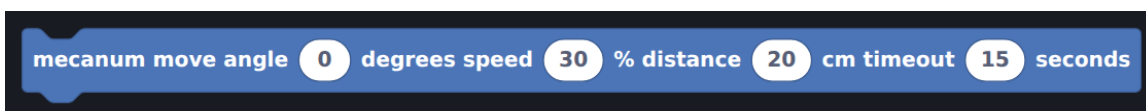


If one wheel turns the wrong way, reverse it with this block before any movement.

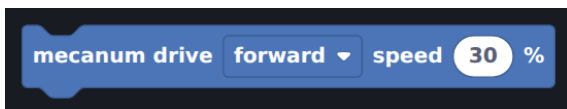
12.3 Moving



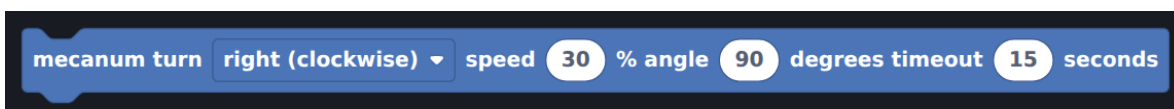
Moves in one of eight directions (forward, forward right, right, backward right, backward, backward left, left, forward left) for a distance, then stops. The program waits.



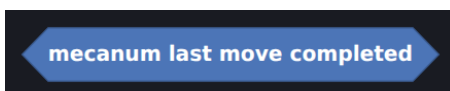
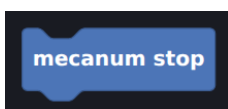
Moves at any angle: 0 forward, 90 right, 180 backward, 270 left.



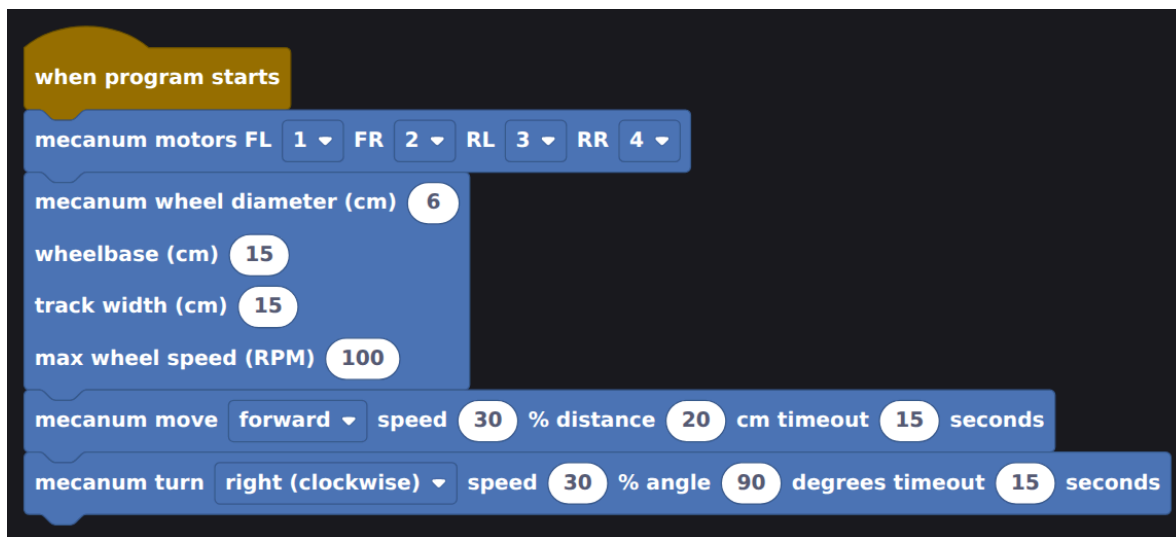
Keeps moving until another move or **mecanum stop**. Use it with the remote control.



Turns on the spot, then checks with the gyro and nudges until it is within half a degree.



true if the last move reached its target; false if it ran out of time. Every move has a **timeout** (15 seconds unless you change it), so a stuck robot does not push forever.



Safety note: test new settings at low speed with the wheels lifted off the table first.

Do not mix **Drive** or **Motors** blocks with Mecanum blocks while the robot is moving: they control the same motors.

Challenges

- **Bronze:** slide right 30 cm, then left 30 cm, back to the start.
- **Silver:** drive a square without turning, so the robot always faces the same way.
- **Gold:** drive a diamond (four diagonal moves) and measure how far from the start the robot finishes. Adjust your chassis measurements to improve it, and record what you changed.

Check yourself

1. Which angle makes the robot slide to the right?
2. Why should you measure your own wheels instead of keeping the defaults?
3. What does **mecanum last move completed** tell you?

Chapter 13: Using blocks strategically

Good programs are not only correct; they react quickly, are easy to tune, are easy to fix and fail safely. This chapter explains the engineering ideas behind choosing one block or structure over another.

13.1 Plan before you build

Write the plan in plain words first (this is called **pseudocode**), then turn each line into blocks:

```
wait for the button
repeat forever:
  if something is closer than 15 cm:
    stop, then turn right
  otherwise:
    drive forward
```

A plan shows mistakes before you have built anything, and each line often becomes one function (Chapter 8).

13.2 Structure: set up once, then loop

Most programs have two parts:

1. **Set-up**, directly under **when program starts**: calibrate the gyro, set up screens and add-ons, set starting values of variables, and often **wait for button press**.
2. **Main loop**: a **repeat forever** that reads sensors, decides, and acts.

Set-up blocks placed inside the loop run again on every pass, which wastes time and can reset things you did not mean to reset.

13.3 Blocks that wait versus blocks that keep going

This is the most important idea in the chapter.

Type	Examples	While it runs
Waits until finished	drive straight, turn, sweep servo, blink LED, wait, mecanum move	The program is paused. No sensors are checked.
Starts and moves on	set drive effort, arcade drive, set drive speed, mecanum drive, set servo angle	The program carries on at once. The robot keeps moving until told otherwise.

- For a fixed route (a square, a dance), waiting blocks are simplest and most accurate.

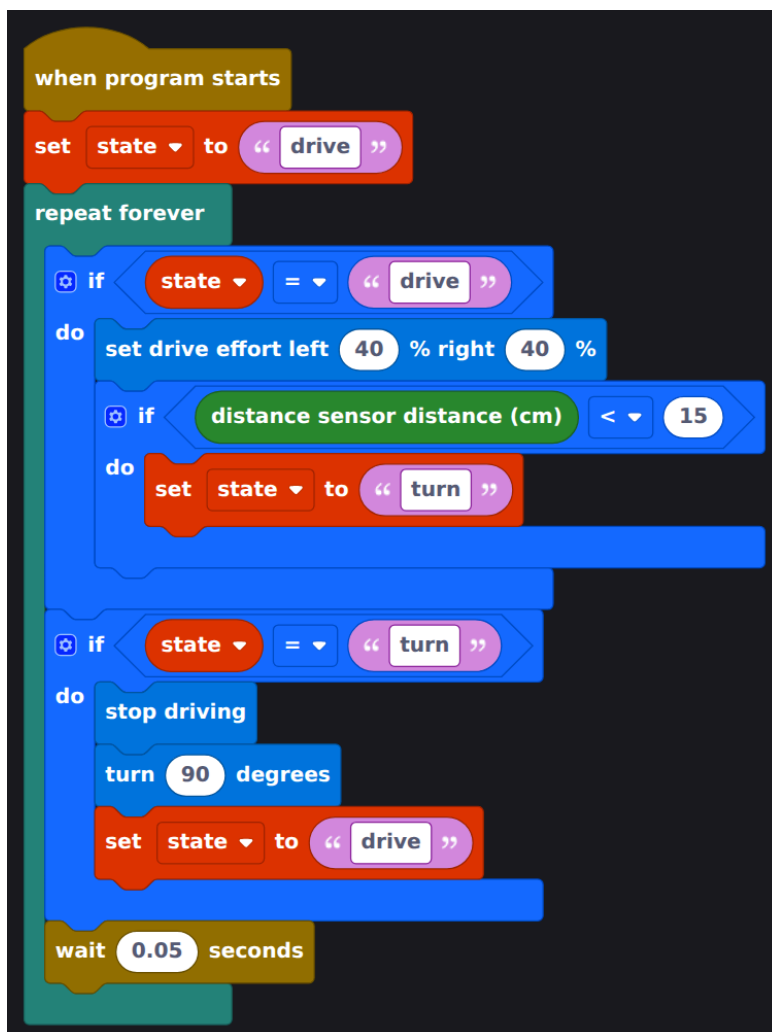
- For reacting to sensors (obstacles, lines, a remote), use keep-going blocks inside a loop, so the robot checks its sensors many times a second.
- A **drive straight 100 cm** inside a sensor loop is a common mistake: the robot cannot notice an obstacle until the whole 100 cm is done.

13.4 Keep loops short, with a small wait

Put a short **wait** (0.02 to 0.05 seconds) at the end of a sensor loop. Several blocks, such as **remote update** and the scrolling-text blocks, are designed to be used this way. Long waits make the robot slow to react: a robot moving at 30 cm/s travels 15 cm during a half-second wait.

13.5 State machines: robots with modes

Many robots behave in **modes**: searching, approaching, turning, finished. Keep the current mode in a variable called `state`, and let each pass of the loop do what that mode needs and decide whether to change mode:



This is called a **state machine**. It keeps complicated behaviour organised: to add a new behaviour, add a new state. Draw the states as boxes with arrows between them before you build.

13.6 Proportional control and tuning

Chapter 5 showed how to react **in proportion** to an error. The same pattern works for many tasks:

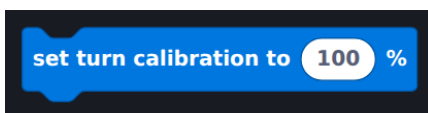
Task	Error	Correction
Keep a distance	distance - target	forward power
Follow a line smoothly	left sensor - right sensor	how much to steer
Face a direction	target heading - heading	how fast to turn

Tune the gain in small steps. Start low, double it until the robot wobbles, then go back to about half of that value.

13.7 Effort, speed and distance

Block style	Controls	Good for
effort %	Power sent to the motor	Simple reactions. Real speed changes with battery level and floor.
set drive speed (cm/s)	A steady speed, corrected using the encoders	The same pace on any battery
drive straight (cm)	An exact distance, holding its heading with the gyro	Planned routes

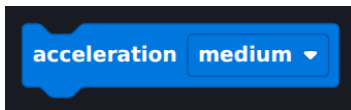
13.8 Tuning turns and starts



If turns are always slightly too far or too short: turn 360 degrees four times (1 440 degrees), measure how far the robot really turned, and use

calibration = $100 \times 1440 \div \text{measured angle}$

Example: the robot turned 1 500 degrees, so use $100 \times 1440 \div 1500 = 96$. Too far gives less than 100; too short gives more.



If the wheels spin or the robot jerks at the start, choose a slower acceleration.

13.9 Gyro: yaw or heading?

- **Yaw** keeps counting (for example 450 after one and a quarter turns right). Use it to measure how far the robot has turned in total.
- **Heading** wraps between 0 and 360, like a compass. Use it to face a direction.
- Always calibrate and zero the gyro in the set-up part, with the robot still.

13.10 Numbers in lists

A list item added through the default text box is **text**, even if it looks like `0.5`. Python treats text and numbers differently, so a block expecting a number can fail with a `TypeError`. The **wait** block accepts number-like text, but other blocks may not. Always use a number block from **Math** when adding numbers to a list.

13.11 Make programs easy to tune

- Put values you will adjust (speed, distances, sensor limits, gains) in variables set at the top. One change then tunes the whole program.
- Put repeated steps in a function.
- Give variables and functions clear names, such as `obstacle limit` rather than `x`.

13.12 A method for finding mistakes

1. **Predict:** say what you expect the robot to do before you click **Run**.
2. **Observe:** watch closely, and read the Console reports and any error.
3. **Narrow it down:** switch parts of the program off with `Alt + click`, or add prints, until you find the block where the prediction and reality split.
4. **Change one thing at a time**, then test again. Changing three things at once hides which one mattered.
5. **Record** what you changed and what happened.

13.13 Tools that help while testing

Tool	How to use it
Watch this value	Right-click a variable or list block (or press <code>W</code>), click Run , then hover over its round block to see the live value
Console reports	Every drive and turn reports how accurate it was

Tool	How to use it
Python tab	Shows the real code your blocks create
Alt + click	Switches a block off without deleting it
Delete unused blocks	Right-click the workspace to remove loose blocks
Pin to toolbox	Right-click a block you use often to add it to Pinned

13.14 Fail safely

- Every keep-going block needs a planned stop: a **wait** then **stop driving**, or **stop driving** in the **else** part of an **if**.
- Use **wait for button press** at the start of deployed programs.
- With a remote, stop the robot whenever the controller is not connected.
- Use **constrain** to keep calculated motor power within safe limits.

Check yourself

1. Your obstacle avoider uses **drive straight 50 cm** in its loop and crashes into walls. Explain why, and fix it.
2. Your robot turned 1 400 degrees when asked for 1 440. What calibration value should you use?
3. Sketch the states for a robot that searches for an object, drives to it, then flashes a light.
4. Why should you change only one thing at a time when testing?

Chapter 14: Projects and challenges

These projects bring several chapters together. Each one lists what it must do, the chapters it uses, hints and ways to extend it. Work in pairs, use the design process below, and keep a project log.

14.1 The engineering design process

1. **Understand:** what exactly must the robot do? How will you know it succeeded?
2. **Plan:** write pseudocode or draw the states (Chapter 13). Decide which sensors and blocks you need.
3. **Build:** one part at a time, as functions.
4. **Test:** predict, run, observe. Record the results.
5. **Improve:** change one thing at a time, and test again.
6. **Present:** explain what works, what does not yet, and why.

14.2 Project log

Keep one line per test in a table like this:

Test	What I changed	What I expected	What happened	Next step
1	First version	Stops 15 cm from the wall	Stopped at 8 cm	Stop earlier: try 20 cm
2	Limit 15 to 20 cm	Stops about 15 cm away	Stopped at 14 cm	Keep, now test turning

14.3 Project list

Project	Must do	Chapters	Level
Robot artist	Draw a triangle, square and hexagon from one function with a sides input	3, 6, 8	Bronze
Parking assistant	Drive towards a wall; RGB LED green, then yellow, then red as it gets closer; stop 5 cm away	2, 5, 6	Bronze
Traffic-light crossing	Wait at a paper "crossing" until the button is pressed, show the light sequence, then cross	2, 3, 5	Bronze
Obstacle course	Get from start to finish through a set of boxes without touching any	3, 5, 6, 13	Silver

Project	Must do	Chapters	Level
Line-following race	Complete a lap of a taped track as fast as possible without losing the line	5, 6, 13	Silver
Stay in the ring	Drive around inside a black-edged ring for 60 seconds without leaving it	5, 6, 13	Silver
Delivery robot	Follow a line to a drop-off point, release a load with a servo, return	4, 5, 8, 13	Gold
Smart car	Headlights, brake lights and indicators with Triggers, while driving a set route	3, 10	Gold
Remote soccer	Two robots, remote-controlled, push a ball into a goal	11, 13	Gold
Room mapper	Turn on the spot in steps of 30 degrees, record the distance at each step in a list, then print the closest direction	5, 7, 13	Gold

14.4 Hints

- **Parking assistant:** use **else if** for the three colour zones, and a proportional speed so the robot slows as it gets close.
- **Obstacle course:** a state machine with states `drive`, `look`, `turn` is easier to extend than one big **if**.
- **Line-following race:** calibrate first. A smoother follower uses both sensors: steer by the difference between left and right readings.
- **Stay in the ring:** when either line sensor sees black, back up, turn a random angle, and carry on.
- **Delivery robot:** write and test one function per stage (`follow line to end`, `drop parcel`, `go home`) before joining them.
- **Room mapper:** use **count with** from 1 to 12, a list, and **turn 30 degrees** after each reading.

14.5 Assessing your project

Criterion	Developing	Achieved	Excellent
Works	Some parts work	Meets every "must do"	Also handles unexpected situations
Program design	One long stack of blocks	Uses loops and variables sensibly	Clear functions or states, easy to tune

Criterion	Developing	Achieved	Excellent
Testing	Few tests recorded	A log of tests and changes	Tests planned to prove each part works
Safety	Robot can run away	Planned stop in every case	Fails safely even when a sensor or remote fails
Explanation	Can say what it does	Can explain how it works	Can explain why it was designed that way

Chapter 15: From blocks to Python

Every block program is turned into real **Python** code, which the robot runs using **MicroPython**, a version of Python made for small computers. Reading that code is the first step to typing programs yourself.

15.1 Where to find the code

Open the **Python** tab at the bottom of the screen. It updates as you move blocks. At the top you will see `import` lines, which load the robot's libraries, and often some long helper code that XRPBlocks adds for accurate driving. **Your program is at the bottom.**

15.2 Blocks and their Python

Block	Python
turn LED on	<code>board.led_on()</code>
wait 1 seconds	<code>time.sleep(1)</code>
print "Hello"	<code>print('Hello')</code>
drive straight 30 cm	<code>xrp_straight(30)</code> (a helper XRPBlocks adds)
turn 90 degrees	<code>xrp_turn(90)</code> (a helper XRPBlocks adds)
stop driving	<code>drivetrain.stop()</code>
distance sensor	<code>rangefinder.distance()</code>
line sensor left	<code>reflectance.get_left()</code>
gyro heading	<code>imu.get_heading()</code>
wait for button press	<code>board.wait_for_button()</code>
repeat forever	<code>while True:</code>
if ... else	<code>if ...: and else:</code>
set count to 0	<code>count = 0</code>
change count by 1	<code>count = count + 1</code>

15.3 Reading a real program

The obstacle avoider from Chapter 6 ends like this:

```

while True:
    if rangefinder.distance() < 15:
        drivetrain.stop()
        xrp_turn(90)
    else:
        drivetrain.set_effort(xrp_effort(40), xrp_effort(40))
        time.sleep(float(0.05))

```

Things to notice:

- **Indentation** (the spaces at the start of a line) shows which lines are inside the loop or the **if**. In blocks, that is the C-shaped block wrapping around others.
- A **colon** : ends the line that starts a loop or a decision.
- `xrp_effort(40)` turns 40% into the 0.4 that the motor library expects.

The counter from Chapter 7 becomes:

```

count = 0
for count2 in range(5):
    count = count + 1
    print(count)
    time.sleep(float(1))

```

`range(5)` counts five times. The loop needs its own counter, so XRPBlocks invents the name `count2`.

15.4 Next steps

- Predict the Python before you open the tab, then check.
- Change a block and watch which line changes.
- When you are ready to type code, the same libraries (`drivetrain`, `rangefinder`, `reflectance`, `imu`, `board`) are used in text-based XRP programming. Your teacher will show you which editor to use.

Check yourself

1. Which Python line matches **repeat forever**?
2. What does the indentation show in Python?
3. Why does `xrp_effort(40)` appear instead of just `40`?

Chapter 16: Troubleshooting

When something goes wrong, check the simple things first: power, cable, connection, then the program. Use the method in section 13.12.

16.1 Connecting and running

Problem	Likely cause	What to try
The robot is not in the Connect list	Robot off, charge-only USB cable, or wrong browser	Switch on, try another cable, use Chrome or Edge
Run says to connect the robot first	Not connected	Click Connect XRP first
Nothing happens after Deploy	Deploy saves the program but does not run it	Click Run , or switch the robot off and on
The deployed program starts driving at once	No start signal	Add wait for button press at the start
Some blocks are grey	They are not connected to when program starts , or were switched off	Snap them into the program, or Alt + click to switch them on

16.2 Driving

Problem	Likely cause	What to try
The robot never stops	A keep-going block has no stop after it	Add wait and stop driving (Chapter 13)
The robot ignores obstacles	A waiting block is inside the sensor loop	Use keep-going blocks in the loop
Turns are too short or too long	Batteries, floor, or calibration	Check batteries, then use set turn calibration
Wheels spin or the robot jerks at the start	Acceleration too fast for the floor	Choose a slower acceleration
Console says a drive or turn "gave up"	The robot was stuck or blocked	Clear the path; check nothing rubs on the wheels
The robot restarts when a servo moves	Batteries cannot supply the current	Charge or replace batteries; move the servo while standing still

16.3 Sensors and add-ons

Problem	Likely cause	What to try
Gyro angle is a large number at the start	Gyro not zeroed	gyro calibrate , then set gyro yaw to zero
Gyro drifts while standing still	Robot moved during calibration	Calibrate again with the robot completely still
Line follower loses the line	Threshold wrong for this floor, or too fast	Recalibrate (Chapter 5) and slow down
Distance reading jumps	Soft or angled object, or noise	Test with a flat, hard object; average several readings
An add-on says it is not answering	Loose Qwiic cable or wrong address	Switch off, re-seat the cable, check the address in the set-up block
Remote page will not open	Phone not on the robot's Wi-Fi, or wrong address	Join XRP-Remote ; use the address printed in the Console

16.4 Program errors

Problem	Likely cause	What to try
A list program crashes with a <code>TypeError</code>	Numbers were added to the list as text	Use number blocks when adding numbers (Chapter 13)
A variable seems to have the wrong value	It was not set before use, or is changed somewhere unexpected	Watch it (Chapter 7) and set it at the start
Strange, random behaviour	Weak batteries	Replace or recharge the batteries

Read the **Console** after a crash. The last lines usually name the problem and the line of Python where it happened.

Still stuck? Save your program, write down what you expected and what actually happened, and ask your teacher.

Glossary

Term	Meaning
Address (I2C)	A number that identifies a device on the Qwiic connector, such as 0x3C
Block	A piece of program you drag and snap together
Boolean	A value that is either true or false
Calibrate	Measure and correct a sensor or movement so it is accurate
Circumference	The distance around a circle: $\pi \times \text{diameter}$
Console	The panel showing messages from the robot
Constrain	Keep a number between a low and a high limit
Deploy	Save the program on the robot as <code>main.py</code> , to run on its own at power-on
Device library	A set of extra blocks and a driver for an add-on part
Differential drive	Steering by turning two wheels at different speeds
Effort	How much power a motor gets, from -100% to 100%
Encoder	A sensor that counts how far a wheel has turned
Error	The difference between where the robot is and where it should be
Feedback	Measuring the result and correcting, over and over
Function	A named group of blocks you can use many times
Gain	How strongly a proportional controller reacts to the error
Gyro (IMU)	The sensor that measures turning and tilt
Heading	A compass direction from 0 to 360 degrees
I2C	The two-wire connection used by Qwiic devices
Input (function)	A value passed into a function when it is called
Loop	Blocks that repeat
MicroPython	The version of Python that runs on the robot
Proportional control	Correcting in proportion to the size of the error
Pseudocode	A program plan written in plain words
Qwiic	A small four-wire plug for add-on parts

Term	Meaning
Reflectance	How much light a surface bounces back
Reporter block	A round block that gives a value, such as a sensor reading
Servo	A motor that moves to and holds an exact angle
State machine	A program organised into modes, with rules for changing mode
Threshold	The value that separates two cases, such as black and white
Variable	A named box that stores a value
Yaw	The total angle the robot has turned, which keeps counting past 360